

Learning to solve coding problems

Netflix L5 interview preparation in Go

The reading edition

Worked examples and visual explanations

Data structures. Algorithms. Caches. Rate limiting.
Recommendations. Worked program designs.

Learn how state changes, why an approach works, and when a different structure is needed. No computer required.

Contents

Learning to solve coding problems	3
Chapter 1 - Go and data structure choices	7
Chapter 2 - Hashing arrays and sliding windows	16
Chapter 3 - Sorting binary search intervals and heaps	25
Chapter 4 - Trees graphs and dependencies	34
Chapter 5 - Stacks greedy choices and dynamic programming	43
Chapter 6 - LRU and weighted caches	52
Chapter 7 - Expiration clocks and concurrency	62
Chapter 8 - Rate limiting and rolling windows	72
Chapter 9 - Deduplication counters versions and scheduling	81
Chapter 10 - Recommendation and streaming ranking tasks	90
Chapter 11 - Modeling larger practical programs	100
Chapter 12 - Complete interview case studies	109
Sources and scope	120

Learning to solve coding problems

This is the reading edition of the Netflix L5 Go interview course. It prepares you for the first live technical interview with another senior engineer. You can read it independently on an e-reader, without an editor, a timer, or an AI conversation. The companion conversation edition teaches the same topics through short explanations, questions, feedback, and verbal program design.

The book teaches through explanations, worked examples, diagrams, state tables, and Go code. Each chapter has an explained Go example connecting its invariant to concrete fields, branches, and mutations. Some are complete algorithms; cache, expiration, and scheduling examples isolate a core operation and explicitly state what the surrounding program supplies. You can trace the examples on the page without executing them. The PDF is not a transcript of the tutor's question bank.

What learning an approach means

A memorized solution tells you what to do when the prompt looks familiar. An approach tells you what information a problem needs, which operations are expensive, and what must remain true as the state changes. That lets you recognize when a familiar method fits a new story and when a changed assumption makes it fail.

For example, a queue can support expiration when timestamps arrive in order. It is not enough to remember “time window means queue.” You should understand why expired timestamps form a prefix, why you can stop when the first fresh timestamp appears, and why out-of-order arrival breaks that reasoning. Once you understand those facts, you can apply the idea to rate limits, recent-hit counts, and expiring viewing events.

The recurring method is to define behavior, construct a correct baseline, identify repeated work, choose state that removes that repeated work, and justify what the state means. Then test the argument against a tiny example and a changed requirement. You will see this process across arrays, graphs, caches, and small services.

A useful kind of mental exercise

Imagine a cache with room for two titles. After writing A and then B, B is most recent. Reading A moves A ahead of B. Writing C now evicts B. You can execute this sequence in your head and explain which map entries and list links must change. You are practising implementation reasoning even though you are not typing pointer assignments.

A good verbal implementation is more specific than “use a map and a list.” It says what each field represents, what a hit and a miss do, which helper performs a removal, and how every return path preserves the contract. The worked program designs model that precision. An AI tutor can play the caller or the computer while you supply the next operation.

Reading and spoken reasoning do not prove that a program compiles or that you can express it fluently under an interview timer. Those are separate skills. This book is meant to make time away from your computer useful, without pretending that you are practising syntax while walking, commuting, or reading on an e-reader.

How to read the chapters

Read the explanations in order the first time a topic is unfamiliar. The examples supply their reasoning and results directly. You do not need to answer a question to reach the explanation. On a later pass, cover the next table row and predict it if that helps you check understanding. This is optional; the book remains complete when read straight through.

Read a diagram as a picture of a specific state, not as a complete implementation. Arrows show relationships or direction of processing. Shaded boxes emphasize active or selected state. Labels and the

surrounding explanation carry the meaning even on a monochrome screen.

State tables show what changed after each operation. Unless explicitly stated otherwise, a row is the state after the operation and its required cleanup have completed. Compare neighboring rows: identify the new input, the field that changes, and the reason another field stays unchanged. This makes bookkeeping visible without requiring code execution.

In the cache example, a Get changes recency but not the number of residents. In a weighted overwrite, both recency and the weight total may change. In expiration cleanup, a stale deadline record can disappear while the current cached value survives. These differences reveal the invariants more clearly than a picture of the final result alone.

The program design at each teaching chapter's end assembles the concepts into a complete small program in plain language. Trace the included example if you wish, but the text already states what happens and why. No section depends on your discovering an unstated solution before the book becomes useful.

Small examples are intentional. Holding a long array in working memory is not the skill being taught. When discussing a chapter, ask the tutor to repeat the current state or shorten the example. You should spend your attention on the invariant and the decision, not remembering ten arbitrary IDs.

How to use the chapters in conversation

With this repository available to ChatGPT, ask it to start the course, continue a chapter, or work through a named topic. The tutor uses the separate conversation sessions with their question progression and coaching notes. If you refer to an example from the PDF, it can retrieve the reading chapter too. Diagrams should become short descriptions when you cannot see them. You do not need to copy lesson prompts or use a browser reader.

The tutor should teach a missing idea before testing it, give specific feedback, and vary an assumption after you understand the baseline. You can say “walk through the function structure,” “give me a counterexample,” “explain why the pointer moves,” or “turn this into a spoken coding exercise.” The written answers remain available if you are reading without an AI.

What the book prioritizes

The first five chapters develop representation, hashing, windows, ordering, graph traversal, and reasoning about choices. The next four apply those tools to caches, expiration, concurrency, rate limits, and event services. Chapter ten develops recommendation and ranking exercises. Chapter eleven models larger programs. Chapter twelve contains two complete interview-style case studies with worked answers, also usable as oral mocks.

Your plan.md shaped the practical emphasis. The exercises are original study scenarios, not a verified list of Netflix questions. Sources establish language behavior, platform context, and recommendation concepts; they do not establish what your interviewer will ask. The aim is transferable reasoning across a wide variety of prompts.

There is no required completion time for a chapter. Independent reading is shorter than a conversation with explanations, attempts, corrections, and follow-ups. Progress means that a new example makes sense and that you can defend your decisions, not that a timer has expired.

Chapter 1 - Go and data structure choices

Choose a structure by the operations you need to make cheap. The story around the data is secondary.

From requirements to representation

Imagine a list of recently watched movies. If you only append and iterate, a slice is a natural fit. If you repeatedly ask whether a particular movie has been watched, add a set. If you must remove the least recently accessed movie while promoting arbitrary movies on access, a slice alone makes each promotion expensive. A map plus a doubly linked list answers both identity and recency questions.

A slice gives constant time indexed access and amortized constant time append. Amortized means a sequence of appends has linear total work even though an individual append may allocate and copy the entire backing array. Inserting or deleting in the middle generally shifts elements and costs linear time. A map gives expected constant time lookup, insertion, and deletion; it does not preserve order. Include key hashing cost when keys themselves are long strings.

A stack removes the newest item. Use it for unfinished nested work, depth first traversal, and undo. A queue removes the oldest item. Use it for breadth first traversal or chronological expiration when arrival and expiration order agree. A heap exposes the smallest or largest priority cheaply but does not keep every element globally sorted. A balanced search tree supports ordered updates and predecessor queries; a sorted slice gives simpler binary search when updates can be appended or are infrequent. Go's standard library does not provide a general ordered map.

Need	First candidate	What it does not solve
Membership or key lookup	Map or set	Sorted order, oldest item
Process in arrival order	Queue with head index	Out of order deadlines
Next minimum repeatedly	Min heap	Arbitrary key lookup without an index
Most recent and least recent	Doubly linked list plus map	Earliest expiration
Latest version before a time	Sorted slice per key	Cheap arbitrary historical insertion
Connections and dependencies	Adjacency list	Implicit cycle protection
Prefix lookup	Trie, or sorted strings plus search	Low memory automatically

For listening: identity suggests a map; arrival order suggests a queue; smallest priority suggests a heap; arbitrary recency changes suggest a map and linked list. These are starting hypotheses. The contract and workload decide whether they are appropriate.

Go mechanics that prevent interview bugs

Initialize a map before assigning into it. Reading a missing key yields the value type's zero value, so use the second boolean result when zero is a valid stored value. A set is often a map from keys to empty structs. Map iteration order is unspecified; sort extracted keys when the result must be deterministic. These rules are documented in the [Go specification](#).


```

seen := make(map[string]struct{})
seen["movie-1"] = struct{}{}
_, exists := seen["movie-1"]
delete(seen, "movie-1")
_ = exists

queue := []int{0}
for head := 0; head < len(queue); head++ {
    node := queue[head]
    _ = node // append newly discovered nodes here
}

```

A slice is a view over an array. Copying the slice value does not copy the elements; an in-place sort can change the caller's data. Use an explicit copy when your contract promises preservation. Reslicing a long-lived queue may retain its backing array. Clear pointer slots when discarding values and occasionally compact or use a ring buffer for bounded services. A short interview traversal with integer IDs rarely needs elaborate memory reclamation.

A string is bytes. Indexing returns a byte; ranging decodes runes, and the range index is a byte offset. If the task defines characters as Unicode code points, convert to a rune slice or maintain rune positions carefully. A rune still is not necessarily a human-perceived character: combining marks and emoji sequences can contain multiple runes. Clarify the required unit. ASCII-only tasks can use byte arrays for speed and simplicity. [Go strings, bytes, runes and characters](#).

Use explicit integer widths where constraints demand them. Durations and timestamps in our exercises use integer ticks, and sums that could exceed 32 bits use int64. Ask whether multiplication can overflow. Avoid sentinel values such as minus one if they are legitimate data; return a value and a boolean. Return an error for invalid input when the prompt expects validation, rather than inventing a successful result.

Failure case. A map from movie ID to a single duration cannot preserve the order of individual watch events.

A single value per movie loses event multiplicity and ordering unless the value itself stores a history. Totals need accumulation; the last ten events need chronological records. The correct key is determined by the query, not merely by an available ID.

Complexity is a budget explanation

Let n be events, u distinct users, and k returned results. Say which one your bound uses. A map of queues can hold up to u times a per-user limit, not merely the limit. Sorting n records is typically $O(n \log n)$; scanning every user's queue for one request adds work proportional to all users. Space includes maps, queued stale entries, recursion depth, and output. Distinguish auxiliary space from output space.

To justify an optimization, identify repeated work. If every lookup scans all records, an index removes that repeated scan. If every iteration sorts everything just to find the next minimum, a heap maintains enough ordering. If every recursive branch recomputes the same remaining problem, memoization reuses answers. Understanding the saved work is more durable than memorizing names.

A slower explanation of what a data structure buys you

Think of a data structure as a decision about what work to do now so that later questions become easier. An unsorted slice is easy to build: put each item at the end. You have not organized it by identity or priority, so a later search may inspect every item. A map does more bookkeeping on insertion so that a later lookup can usually jump directly to a small part of the stored data. A sorted slice pays for order so that searches can eliminate whole ranges. None is universally best because they invest work in different capabilities.

A hash map does not magically know where every key lives. A hash function converts a key into a value used to locate storage. Different keys can lead to the same candidate location; this is a collision. The map must still compare keys or use another collision-handling scheme. That is why

expected constant-time lookup is not the same as a promise that every input takes one machine instruction. You usually do not implement the hash table during an interview, but you should understand the guarantee you are relying on.

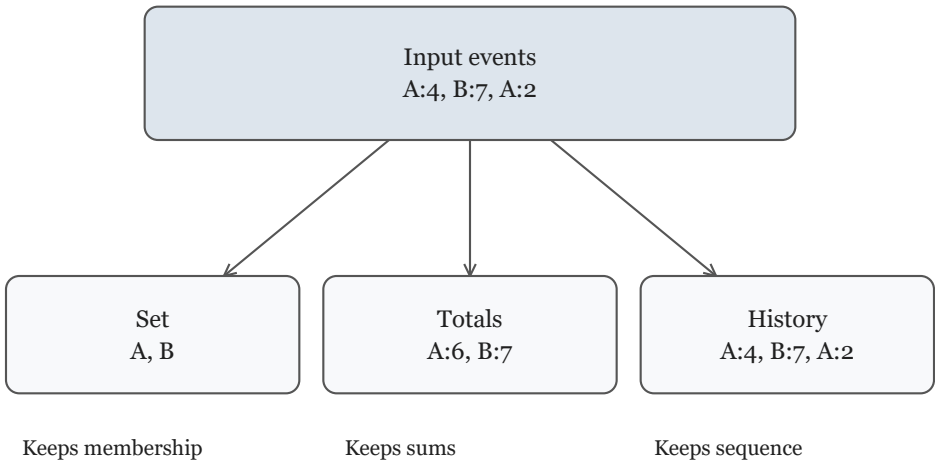
An index also has a maintenance cost. Suppose you keep a slice of movies and a map from movie ID to its slice position. Looking up a movie becomes easy. Now remove the first movie by shifting every later movie one place left. Every shifted position in the index has become wrong. The combined structure must maintain agreement between its two views. This small example foreshadows the central challenge of LRU caches: a fast structure is only useful while its invariants remain true.

Ownership is another part of representation. If two parts of a program hold slices over the same backing array, they share the elements even though each slice has its own length and capacity. Imagine handing someone a window onto a bookshelf, rather than photocopies of the books. Moving the window does not copy books; changing a visible book changes what another overlapping window sees. Appending may allocate a new shelf, so whether changes remain shared can depend on capacity. Correct code should not rely on accidental allocation behavior to provide isolation.

Worked example: one event stream, three different questions

Suppose a viewer produces three events: A for four minutes, B for seven minutes, then A for two minutes. Before choosing a container, write down the answer the caller needs. The same input supports several incompatible summaries.

ONE INPUT, THREE INFORMATION BUDGETS



Each arrow transforms the input; lost detail cannot be recovered.

Three representations of the same viewing events. Only the history retains both order and individual durations.

For total watch time by title, accumulate into a map: A becomes six and B becomes seven. For whether a title has ever appeared, keep only membership: A and B. For the last two events, keep ordered records: B for seven, followed by A for two. All three representations are correct for their own queries. Only the history can answer all three afterward, but it costs more space and may require more work per query.

Representation	Stored state	Information discarded
Membership set	A, B	Counts, durations, order
Totals map	A: 6, B: 7	Individual events and order
Event history	A:4, B:7, A:2	Nothing from these records

This is a useful way to evaluate a proposed optimization. A smaller representation is a claim that some information will never be needed again. If the interviewer changes the query to the last two events, the

totals map cannot be repaired by a clever loop. The missing history must be retained from the start or recovered from another source.

Worked example: deletion changes the cost of an index

A slice stores movie records in the order A, B, C. A second map stores A at index zero, B at one, and C at two. Lookup is now fast, but deleting B exposes a design choice.

If display order must be preserved, shift C left and update C's map position to one. For a longer slice, every shifted record needs its index repaired. Lookup became cheaper by making updates more expensive. If order is irrelevant, replace B with the last record C and shorten the slice. Only C's index changes. Both versions remove B from the map.

The distinction matters in Go because a slice position is not stable identity. A record's index can change even when its movie ID does not. An index map therefore needs an explicit invariant: for every resident ID, the indexed slice position contains that same ID. A test that only looks up A can pass while C's stale index remains broken.

The general lesson is to choose a representation for a workload, not for one isolated operation. A directory with frequent lookups and rare deletions may justify the extra index. A three-record list queried once may be simpler to scan. Both decisions follow from how much repeated work is actually saved.

Putting the program together: an event registry

Suppose we want to accept event IDs while preserving the order in which distinct IDs first appeared. The public operation `Accept` takes an ID and returns whether this call introduced a new ID. `History` returns the unique IDs in first-appearance order. The empty string is a valid ID. Repeated arrivals do not move an ID to the end.

The registry needs two fields: a membership set and an ordered slice. The set answers whether an ID already exists; the slice preserves an

order that the set cannot supply. In Go terms, a struct could contain a map from string to empty struct and a slice of strings. A constructor initializes the map. Every ID in the slice must occur exactly once in the set, and the two structures must have equal sizes.

Accept first looks up the ID. If present, it returns false without mutation. Otherwise it records the ID in the set, appends it to the slice, and returns true. History returns the slice's contents. If callers are allowed to mutate their returned slice, History should return a copy; otherwise a caller could overwrite the registry's internal ordering without updating its set.

On A, B, A, the first two calls return true and the third false. History remains A, B. An empty input history is an empty result, not an error. A test using an empty-string ID catches implementations that confuse a legitimate value with a missing-key sentinel. A test that mutates a returned history catches accidental shared ownership.

Accept is expected constant time, with amortized append cost. Returning a protected snapshot costs linear time in the number of distinct IDs. If the next requirement is Remove, deleting from the set is easy but preserving slice order requires finding and shifting the ID. That new operation changes the workload and may justify another representation.

Go example: preserve zero and missing separately

This complete function copies the record out of a map and returns its presence separately. The caller owns the map; the function does not mutate it. A map can be nil here because reads are allowed even when no storage was allocated.

```
type Movie struct {  
    Title string  
    Minutes int  
}  
  
func LookupMovie(movies map[string]Movie,  
    id string) (Movie, bool) {  
    movie, found := movies[id]  
    return movie, found  
}
```

With A stored as `Movie{Title: "A", Minutes: 0}`, looking up A returns that record and true. Looking up B returns the zero-valued `Movie` and false. Testing `Minutes` against zero would confuse these outcomes. The `:=` declares local variables, while the second result from a map lookup records presence. Returning a struct copies its fields; it does not expose a pointer to the map entry. If a field were itself a slice or map, that field's underlying storage would still be shared.

Expected lookup time is constant for bounded-size keys. The useful design choice is the result contract, not the function's size: callers can distinguish a valid empty value from absence without a special sentinel.

Chapter 2 - Hashing arrays and sliding windows

A faster solution often remembers precisely the information that the next step needs.

Two Sum as a lesson in sufficient state

Suppose playback chunks have sizes two, seven, eleven, and fifteen. Find two distinct chunks totaling nine. Trying every pair is correct and quadratic. When considering seven, the only relevant question about earlier values is whether two occurred. Store value to earlier index. Before inserting the current value, look for target minus current value. This order prevents reusing the same element.

The invariant is that before processing position i , the map contains only positions before i . If the complement exists, the indices are distinct and their values sum to the target. If it does not, storing the current index prepares exactly what future positions need. Duplicates such as three and three with target six work because the first three is stored before the second is examined. The expected time is linear and space is linear.

Reference: `TwoSum` in `algorithms.go`.

Transfer this idea to event matching, reconciling IDs, or counting pairs. Ask whether you need existence, a count, or actual positions. A set cannot answer how many matching records exist; a frequency map can. An index chosen for one query may discard information required by another.

Grouping means inventing a stable identity

To group anagrams, convert each word to a signature. Sorting its letters gives the same signature to words with the same letter multiset. For lowercase English letters, an array of twenty six counts is a comparable Go map key and avoids sorting each word. For a word of length m , sorting costs $O(m \log m)$; counting costs $O(m)$ under the fixed alphabet assumption. A signature that stores only which letters appear would wrongly group “abb” and “ab.”

Real-world analogues include normalizing event identities and deduplicating equivalent requests. Decide what equivalence means first. Case folding, Unicode normalization, and ignoring punctuation are requirements, not automatic improvements. Making a key “more normalized” can merge records that should remain distinct.

Sliding windows maintain a local invariant

Find the longest contiguous sequence of titles with no repeated ID. A brute-force approach starts at every position and extends until a duplicate appears. The overlap between these searches is wasted work. Instead maintain a left boundary and a map of last seen positions while advancing the right boundary once.

Walk through A, B, B, A. After A and B, the window has length two. The second B was at position one, so move left to position two. At the final A, its previous position is zero, already outside the window. Do not move left backward. The update is left equals the larger of current left and previous position plus one. The window now contains B, A, again length two.

The invariant is that the current window contains no repeated ID. Adding the rightmost element can introduce only a repetition of that element. Moving past its last occurrence repairs the invariant with the smallest necessary movement. Both boundaries only move forward, giving linear time. A nested while loop that shrinks the left side can still be linear because each element leaves once. Reference: LongestUnique, using rune positions.

A boundary that must not move backward. In A, B, B, A, assigning left to the previous occurrence plus one without comparing it with the current left boundary is incorrect.

The final A would move left back to one and reintroduce the duplicate B. A correct boundary only advances. Explain the invariant before quoting a max expression.

Know when a window is not enough

For positive numbers, shrinking a sum window reduces its sum, so you can use directional reasoning for certain threshold tasks. With negative numbers, that assumption disappears. For example, the array three, minus two, two contains a length-three subarray summing to three even though an early sum may suggest shrinking. Do not use a positive-only argument on signed data.

To count subarrays with sum k for arbitrary integers, maintain prefix sum p . A subarray ending here sums to k whenever an earlier prefix equals p minus k . Store counts of earlier prefixes, initialized with zero occurring once to represent a subarray starting at the beginning. Add the number of matches before incrementing the current prefix count. For one, minus one, one with target one, the answer is three. This is hashing of accumulated state, not a window. Reference: CountSubarrays.

Prefix and suffix work without division

Product Except Self asks for the product of all values except the current one. Dividing a total product fails at zeros and may violate the prompt. First fill each output position with the product strictly to its left. Walk backward with a running product strictly to its right, multiplying it into the output. On two, three, four, left products are one, two, six; right products supply twelve, four, one; the result is twelve, eight, six.

The empty product is one. That identity makes the edges work without special cases. One zero means only its position may have a nonzero output; two zeros make every output zero. The algorithm is linear with constant auxiliary space excluding output, assuming products fit the

chosen numeric type. The deeper pattern is combining summaries from both sides of a position.

Deriving an algorithm by deciding what the past must remember

Imagine reading a long sequence from left to right. You cannot keep re-reading every earlier element if you want a fast solution. Instead ask: what question will the next element ask about the past? In Two Sum, the next value asks whether its complement appeared earlier. In frequency counting, it asks how many times the same value appeared. In longest unique substring, it asks where its last occurrence was. These are three different summaries of history, even though all can be stored in a map.

This question helps you invent the state rather than guess a named pattern. A map is a container, not the whole solution. You must choose the key, the meaning of its value, and when an update occurs. Storing only presence loses frequency. Storing only the latest index loses all earlier positions. Either loss is fine when those earlier details cannot affect any future answer. That last sentence is a correctness argument: discarded information must be irrelevant to the remaining task.

A window adds a boundary to this reasoning. Your summary describes a current contiguous region rather than the whole prefix. Every time you move the left edge, you must remove that departing element's effect. That is why a count map works for an at-most-two-distinct window. A single boolean would not tell you whether another copy remains inside after one copy leaves.

Do not confuse “contiguous” with “sorted.” A viewing session is contiguous because it occupies consecutive events in the original order. Sorting those events changes the session. Two pointers can sometimes work without sorting, but only if the problem supplies another directional property. For a positive sum, moving the right edge increases the sum and moving the left edge decreases it. Signed values remove that property. The method fails because its proof fails, not because a pattern lookup says negative numbers belong somewhere else.

Prefix sums are a different way of representing a contiguous region. Picture cumulative watch duration as an odometer. The duration between two points is the later reading minus the earlier reading. If the current reading is seven and you want a segment totaling three, you are looking for an earlier reading of four. Store how many earlier readings had each value. Negative durations or adjustments do not break subtraction, so this approach survives where directional window reasoning does not.

Worked example: watch a sliding window repair itself

Find the longest contiguous session containing at most two distinct titles in A, B, A, C, C. The slow baseline starts at every position and scans until a third title appears. Adjacent starts repeat much of the same counting. A moving window reuses those counts.

WINDOW REPAIR: THE LEFT EDGE ONLY ADVANCES

Shaded = active; white = already removed

1 Add C: invalid



2 Remove A



3 Remove B



L = 2

R = 3

Distinct counts: 3 -> 3 -> 2. Only the last row is valid.

When C introduces a third title, two removals repair the window to A, C. The next C can then extend it without another removal.

Keep a left boundary, a right boundary, and a frequency map for the records between them. Adding the first C gives counts A:2, B:1, C:1. Removing the leftmost A is insufficient: A still has count one, so three distinct titles remain. Remove B next and delete its zero-count entry. Now the window is A, C and is valid again.

New title	Window after repair	Counts	Best length
A	A	A:1	1
B	A, B	A:1, B:1	2
A	A, B, A	A:2, B:1	3
C	A, C	A:1, C:1	3
C	A, C, C	A:1, C:2	3

The repair is a loop, not a single conditional. Its stopping condition is that the number of positive counts is at most two. Each record enters once and leaves at most once, so the total number of boundary moves is linear even though one arrival can trigger several removals. Record the best length only after repair; otherwise an invalid window can become the answer.

Worked example: negative numbers change the approach

Now count subarrays whose sum is two in the array 2, -1, 1. A shrinking window based on whether the sum is too large is unreliable because removing a negative number increases the sum. The needed relationship is between prefix sums.

The running prefix sums, including the empty prefix, are 0, 2, 1, 2. A subarray sums to two exactly when its ending prefix is two larger than its starting prefix. Maintain frequencies of earlier prefix sums. Seed zero with frequency one so a subarray starting at the first element can be counted.

At running sum two, look for zero and count the first element. At running sum one, look for minus one and find none. At the final sum two, look for zero again and count the whole array. There are two answers. Insert each current prefix only after looking up the required earlier value; that order prevents counting a zero-length subarray when the target is zero.

The two examples both process an array from left to right, but their proofs are different. The title window can repair a violation by discarding a prefix. Prefix counting instead remembers all earlier boundary values that could complete a valid difference. Recognizing that distinction is more useful than identifying both as map problems.

Putting the program together: a session with at most two distinct titles

The function takes an ordered sequence of title IDs and returns the longest contiguous length containing at most two distinct titles. It preserves input order and returns zero for an empty sequence. “Contiguous” rules out sorting. We need counts within the active window, a left index, and the best length seen so far.

For each right index, increment the incoming title's count. While the count map contains more than two distinct keys, decrement the title at the left boundary, delete its map entry if the count becomes zero, and advance left. Once the window is valid, compare its length with the best answer. After the final right index, return the best length.

The shrinking loop is essential. One removal may not eliminate a distinct title if another copy remains. For A, A, B, C, adding C creates three distinct titles. Removing the first A leaves another A, so the window is still invalid. Removing the second A leaves B, C and restores the rule. The best length was three, from A, A, B.

The map invariant is exact counts of titles between left and right, inclusive. The validity invariant is checked after shrinking. Left never moves backward, so each event is added once and removed at most once.

Total work is linear; the count map holds at most three keys during the immediate repair step for this fixed distinct-title bound.

To return the actual interval, store the best starting position whenever a new best length is found. Define ties: retaining the first best found returns the earliest longest interval. This extension changes output bookkeeping without changing the window invariant. Useful mental tests are an empty sequence, all one title, A-B-A, and A-B-C-B. They exercise absence, duplicates, a valid repeated title, and necessary shrinking.

Go example: repair the window before measuring it

This complete function implements the earlier at-most-two-title example. `count` contains only positive frequencies, so its length is exactly the number of distinct titles in the window. The input slice is read without mutation.

```
func LongestTwo(titles []string) int {
    count := make(map[string]int)
    left, best := 0, 0
    for right, title := range titles {
        count[title]++
        for len(count) > 2 {
            old := titles[left]
            count[old]--
            if count[old] == 0 {
                delete(count, old)
            }
            left++
        }
        length := right - left + 1
        if length > best {
            best = length
        }
    }
    return best
}
```

For A, B, A, C, C, the result is three. On the first C, the inner loop removes A once and then B; a single `if` would leave the window invalid. Deleting a zero count is equally important: leaving a zero-valued B entry would make `len(count)` overstate distinct membership. Empty input returns zero because the outer loop never runs.

Each index enters and leaves at most once, giving expected linear time. With a fixed two-title limit the map holds at most three distinct keys during repair, so its auxiliary entry count is constant. This bound relies on deleting zero counts.

Chapter 3 - Sorting binary search intervals and heaps

Ordering can turn a global search into a local decision. Decide whether you need full order, a boundary, or just the next best item.

Two pointers after sorting

For Three Sum, find distinct value triples whose sum is zero. The cubic baseline tries all triples. Sort first, then fix one position and search the remaining suffix with a left and right pointer. If the sum is too small, moving right inward cannot increase it; move left forward. If it is too large, move right backward. Each pointer move rules out a family of impossible pairs.

For minus four, minus one, minus one, zero, one, two, fixing minus one allows the pairs minus one with two and zero with one. Skip repeated fixed values and repeated pair values after emitting a triple. The result is unique by values, not by indices. Sorting loses original positions unless you retain them. The full complexity is quadratic time after sorting, with output space proportional to the number of triples; copying to avoid input mutation adds linear space.

Why the pointer move is justified. When a pair sum is too small in a sorted array, keeping the left value and moving the right pointer inward cannot increase the sum.

With the same left value, every smaller right index gives a value no greater than the current right value, so none can fix a sum that is already too small. Sorted order supplies the comparison. “Because that is the pattern” is not a proof.

Merge intervals by remembering one frontier

Suppose sessions occupy half-open intervals: start included, end excluded. Sort by start. Keep a current interval; if the next start is less than its end, the intervals overlap, so extend the end to the larger end. Otherwise emit the current interval and begin another. Once a gap is found, no later interval can bridge backward across it, because later starts are no smaller.

For two to five, one to three, and seven to nine, sorting gives one to three, two to five, seven to nine. The first two merge into one to five. The gap before seven finalizes that answer. If touching intervals should merge as continuous coverage, use less than or equal instead of strictly less. For concurrent viewer counts, touching half-open intervals do not overlap: a departure at time five is processed before an arrival at time five.

A sweep line generalizes merging to “maximum simultaneous sessions.” Turn starts into plus one events and ends into minus one events, sort by time with the agreed tie order, and track a running count and maximum. Merging answers union coverage; sweeping answers overlap counts. Reusing one algorithm without noticing this distinction is a common modeling error.

Binary search asks for the first true position

Binary search requires a monotone predicate: false, false, then true for the rest. To find the latest metadata version at or before time fifteen in timestamps ten, twenty, thirty, search for the first timestamp greater than fifteen. That is index one; the answer is the previous entry, index zero. If the first true index is zero, there is no suitable version. If no index is true, all versions qualify and the final one is the answer.

```
i := sort.Search(len(versions), func(i int) bool {
    return versions[i].Time > queryTime
})
if i == 0 { return "", false }
return versions[i-1].Value, true
```

Go's `sort.Search` returns the first true index or the length, not minus one. The important skill is defining the predicate and handling both edges. For a hand-written version, use a half-open search range and compute middle as low plus half of high minus low. Every iteration must shrink the unresolved range.

In a rotated sorted array with distinct values, at least one half around the midpoint is sorted. Determine which half, check whether the target belongs in its value range, and discard the other. With duplicates, values at both ends and midpoint may fail to identify a sorted side; discarding equal endpoints can degrade to linear time. Do not promise logarithmic worst-case behavior under an assumption you have removed.

A heap keeps only the ordering you need

A binary min heap is usually stored in a slice representing a complete binary tree. The root is index zero. For a node at index i , its children are at two times i plus one and two times i plus two; its parent is at integer division of i minus one by two. The invariant is that each parent is no greater than either child. This guarantees the root is smallest, but siblings and distant branches need not be ordered.

Insert at the end and move upward while the parent comparison is violated. To remove the minimum, move the last item to the root and repeatedly swap downward with the smaller child until the invariant holds. The height is logarithmic, so insertion and removal cost logarithmic time. Inspecting the root is constant time. Building a heap from a whole slice by repairing internal nodes bottom-up takes linear time; inserting every element separately costs $O(n \log n)$. Try drawing the slice three, eight, five, ten, nine: it is a valid heap even though eight appears before five.

To return the top k frequent titles, first count occurrences with a map. Sorting all u unique titles costs $O(u \log u)$ and is a fine baseline. If k is small, maintain a heap of at most k winners whose root is the worst retained candidate. A new candidate worse than the root cannot belong in the top k ; a better one replaces it. The invariant is that after any processed prefix, the heap contains that prefix's best k candidates.

The complexity is expected $O(n)$ for counting and $O(u \log k)$ selection, followed by $O(k \log k)$ if output must be ranked. Define deterministic ties, such as lower title ID winning equal counts. That means the heap's worst element has lower count, or a lexicographically larger ID at the same count. A reversed tie comparison silently returns the wrong boundary item. When k is one, treat the heap step as constant work.

Go's heap interface supplies length, comparison, swapping, appending, and removing the final slice item. Your type's `Pop` method removes the final element; `heap.Pop` rearranges the root there first. Calling the type's `Pop` directly bypasses heap repair. Changing a priority requires `heap.Fix` with a maintained index, or a fresh versioned record that makes old records stale. [Go heap documentation](#).

Why order lets you stop looking

An efficient search is often built around a justified refusal to inspect some candidates. In an unordered list, discovering that one number is too small says little about the others. In a sorted list, that observation tells you about an entire region. The sorting step creates evidence that later decisions can exploit. When explaining an ordering algorithm, identify which candidates a step rules out and why they cannot contain the answer.

Binary search is not restricted to finding an exact value. Its deeper task is locating a boundary between false and true. Suppose versions exist at times ten, twenty, and thirty and you query time twenty. The predicate “version time is greater than twenty” is false, false, true. The first true version is too new; the previous version is the newest allowed one. The exact-match problem disappears into a boundary definition. This same

reasoning supports first capacity that succeeds, first invalid position, and insertion points, provided the predicate really is monotone.

A heap deliberately creates less order. Think of a tournament where you know the current weakest retained winner, but you have not ranked every contestant. To select the best two candidates, compare each newcomer with that weakest winner. If the newcomer loses, it cannot enter the best two. If it wins, replace the weakest and repair the tournament. The complete order is unnecessary until the final two need to be printed.

Sorting and a heap are therefore not rival tricks with one always superior. If you need all results ranked, sorting is direct. If you only need a few winners from a large collection, maintaining a small boundary can save work. If you repeatedly update priorities and ask for the next task, a heap can retain useful state between calls. But if priorities change without heap repair, the stored comparisons become stale: the old tournament result is no longer evidence of today's winner.

Worked example: binary search locates a boundary

A title's metadata versions were saved at times 3, 7, and 12. A query at time 9 needs the version at 7. Searching for an exact timestamp would miss the answer. Search instead for the first stored time greater than 9, then step back once.

BINARY SEARCH FINDS THE FIRST TRUE POSITION

Predicate: timestamp > 9



RETURN index 1

STOP at index 2

Find the boundary first; return its predecessor.

For a query at time nine, the first timestamp greater than the query is twelve. Its predecessor, seven, is the answer.

Use the half-open search interval from low inclusive to high exclusive. Initially low is zero and high is three. The midpoint is one, containing seven. Seven is not greater than nine, so the first greater value must be to its right: low becomes two. The next midpoint is two, containing twelve. Twelve is greater than nine, so high becomes two. The boundaries meet at index two, and index one supplies the answer.

Query time	First greater position	Returned version
2	0	Missing: no predecessor
7	2	Time 7
9	2	Time 7
20	3, past the end	Time 12

Writing the predicate in words makes the boundary cases follow from the same algorithm. An empty history has boundary zero and no predecessor. A query after every version has boundary equal to the

history length and returns the last version. If equal write timestamps are allowed, the contract must also say which version wins; replacing the equal-time record before searching keeps the history unambiguous.

Worked example: a heap keeps only useful order

Keep the three largest values seen in 4, 9, 2, 7, 5. A sorted list of every observation would work, but the query never asks for the complete order. Maintain a min heap containing only the three current winners. Its smallest member is the one most vulnerable to replacement.

Incoming value	Retained values, shown sorted	Decision
4, then 9, then 2	2, 4, 9	Fill three slots
7	4, 7, 9	Replace the root 2
5	5, 7, 9	Replace the root 4

The sorted display in this table is for the reader. The heap itself guarantees only that its root is the smallest retained value and that every parent is no larger than its children. Sorting the three winners at the end produces presentation order. The incoming value needs comparison only with the weakest winner: if it cannot beat that one, it cannot enter the top three.

For movie records, the comparison must include the tie rule too. If smaller IDs win equal scores, the weakest winner among tied scores is the larger ID. A score-only heap can return an inconsistent subset even if a final sort makes that subset look orderly.

Putting the program together: a time-based value store

Expose `Set(key, value, timestamp)` and `Get(key, queryTime)`. Each key has its own ordered history. Writes must be nondecreasing for that key; an earlier timestamp is rejected, and a write at the same timestamp replaces the latest value. `Get` returns a value plus a found flag. An empty stored string is different from no suitable version.

The store has a map from key to a slice of version records. Each record contains a timestamp and value. `Set` finds that key's history. If it is empty, append the new version. Otherwise compare with the final timestamp: reject a smaller timestamp, replace the final value if equal, or append if larger. Reject before mutating so a failed call preserves the sorted-history invariant.

`Get` finds the history and searches for the first version whose timestamp is greater than the query. If that position is zero, return missing. Otherwise return the version immediately before it. If the search finds no greater version, its returned position is the history length, and the final version is the answer. A helper called `firstAfter` can isolate that boundary-search responsibility from the public method.

With quality at time ten equal to HD and at twenty equal to UHD, `Get` at fifteen returns HD. At nine it returns missing; at twenty it returns UHD; at thirty it still returns UHD. Replacing the time-twenty value with HDR changes the last three relevant answers without adding a duplicate timestamp. Attempting a time-twelve write afterward is rejected under this contract.

`Set` is amortized constant time for ordered writes. `Get` is logarithmic in the versions for the requested key, plus expected map lookup. Allowing arbitrary historical insertion changes `Set`: a sorted slice needs a search and potentially linear shifting. The new write contract should not silently invalidate `Get`'s binary-search precondition.

Go example: implement the predecessor search

The version timestamps must be strictly increasing; equal-time writes have already been resolved. This complete binary search narrows unresolved elements in $[lo, hi)$. When that interval is empty, lo is the first greater position, possibly the slice length.

```
type Version struct {
    At int64
    Value string
}

func VersionAt(v []Version, at int64) (string, bool) {
    lo, hi := 0, len(v)
    for lo < hi {
        mid := lo + (hi-lo)/2
        if v[mid].At <= at {
            lo = mid + 1
        } else {
            hi = mid
        }
    }
    if lo == 0 {
        return "", false
    }
    return v[lo-1].Value, true
}
```

For timestamps 3, 7, 12 and query nine, the first branch discards timestamps through seven. The second keeps twelve as the candidate boundary. The answer is the preceding record at seven. Empty input and a query before the first write both finish with lo equal to zero and return missing. A query after the final write finishes with lo equal to the slice length and returns its final record.

The explicit $lo == 0$ guard prevents an index of minus one. There is no separate exact-match branch: the less-than-or-equal comparison sends an equal timestamp into the eligible prefix. Time is logarithmic in the number of versions; auxiliary space is constant.

Chapter 4 - Trees graphs and dependencies

A graph problem asks how information moves through relationships. Make vertices, edges, and visitation rules explicit.

Recognizing the graph hiding in a prompt

Movies linked by shared actors form a graph. Encoding jobs that depend on source assets form a directed graph. Adjacent land cells form an implicit graph. A filesystem directory structure is a tree if every node has one parent and links cannot create cycles. Start by defining the vertices and whether edges are directed, weighted, or generated from neighboring coordinates.

An adjacency list stores each vertex's neighbors. It uses $O(V \text{ plus } E)$ space and supports traversals in $O(V \text{ plus } E)$ time. An adjacency matrix uses quadratic space but gives constant time edge existence queries. Sparse interview graphs usually favor lists. A grid already represents its vertices; you do not need to allocate an explicit object for every edge.

Breadth first search and shortest hops

To find the fewest recommendation links from one movie to another, enqueue the start and mark it discovered. Pop in first-in-first-out order. Enqueue unvisited neighbors with distance one greater. The queue processes all distance-zero nodes, then distance-one nodes, then distance-two nodes. Therefore the first discovery of a vertex has the shortest number of edges.

Mark on enqueue, not on dequeue, so two parents do not add the same vertex repeatedly. Keep a parent map if you must reconstruct the actual path. An empty queue before reaching the destination means unreachable. Breadth first search finds shortest paths when each edge has equal cost. If edges represent different download durations, shortest hops may not mean shortest time.

For nonnegative weighted edges, Dijkstra's algorithm uses a min heap of tentative total distances. When removing a heap record, skip it if its distance is no longer the best recorded one. Relax each outgoing edge by checking whether the current distance plus its cost improves the neighbor. With stale heap entries, a typical bound is $O((V \text{ plus } E) \log E)$, often written $O((V \text{ plus } E) \log V)$ for simple graphs. Negative edges break the greedy finalization argument. Dijkstra is a useful optional extension, not the first implementation to master.

Failure case. A one-edge route costing ten is more expensive than two edges costing one each.

The direct route has one edge but cost ten. A to C to B has two edges and cost two. State whether the task minimizes edge count or total weight before choosing a traversal.

Depth first search and connected components

Number of Islands asks how many connected land components exist. Scan every cell. On an unvisited land cell, increment the component count and explore all reachable land with a stack or recursion. Mark visited before pushing neighbors. The outer scan finds component entry points; the traversal prevents counting the same island twice. Clarify four-way versus diagonal adjacency and whether mutating the grid is allowed.

For a large grid, an explicit stack avoids relying on deep recursion. Each cell is visited once and each has a constant number of neighbors, so time is $O(\text{rows times columns})$. A separate visited structure uses the same order of extra space. Mutating land to water avoids a separate visited map but the traversal stack can still grow linearly.

Tree invariants can be global

Validating a binary search tree by comparing each node only with its children is insufficient. A value of twelve can be the right child of a five inside the left subtree of a ten: its local relationship to five is fine but its relationship to ten is wrong. Pass allowable lower and upper bounds down the recursion, or perform an inorder traversal and verify strictly increasing values when duplicates are forbidden.

Inorder means visit left subtree, then node, then right subtree. Keep a previous value and a boolean saying whether it exists; avoid using the smallest integer as an uninitialized sentinel. A tree traversal costs $O(n)$ time and $O(h)$ call-stack space, with h potentially n for a skewed tree. Level order traversal is simply BFS with either a captured queue length per level or explicit distances.

Dependencies and cycles

For job prerequisites, direct each edge from prerequisite to dependent. Count each vertex's incoming edges. Put all zero-indegree vertices in a queue; remove one, output it, and decrement its dependents. A dependent becomes ready when its indegree reaches zero. If you output fewer than V vertices, the remaining subgraph contains a cycle. This is Kahn's topological sort.

With A before C and B before C , A and B may appear in either order, but C must follow both. If deterministic output is required, use a min heap of ready IDs rather than relying on map iteration. Clarify whether duplicate edges are distinct requirements or should be deduplicated. A consistent adjacency list and indegree count can accommodate duplicates, but mismatching their treatment is a bug.

To clone a graph, memoize original pointer to clone pointer. Create and register the clone before recursively cloning neighbors, otherwise a cycle causes infinite recursion. Reuse that clone whenever the original is encountered again. A map keyed only by a displayed label is unsafe if different vertices may share labels.

Think about the frontier before thinking about recursion

Every traversal has discovered territory and territory still waiting to be explored. The waiting territory is its frontier. Breadth first search chooses the oldest discovered work next; depth first search chooses the newest unfinished work next. That small policy difference changes the order in which facts become known. Neither traversal needs a particular product story: movies, cities, cells, and jobs can all be vertices.

Consider A connected to B and C, with B connected to D. Breadth first search discovers B and C one step from A before it explores D two steps away. The queue preserves layers. Depth first search might follow A, B, D before returning to C. That is useful when exploring an entire branch, but the first route it finds is not necessarily the shortest route in edge count. To choose between them, identify the property you need from the discovery order.

Visited state is not just a performance trick. It gives a meaning to discovery. Without it, two nodes pointing at one another can generate work forever. For a queue traversal, marking a node when it enters the queue means it has already claimed a place on the frontier. Waiting until removal allows multiple parents to enqueue it. In a graph-cloning problem, a similar early registration says that an original node already has a clone identity, even if that clone's neighbors are not fully built yet.

Dependency processing uses a different notion of readiness. A discovered job is not necessarily runnable. It becomes ready only after all its prerequisite edges have been accounted for. Indegree records the number still blocking it. Removing a ready job reduces that count for its dependents. If progress stops while unresolved jobs remain, you have a structural obstruction: every remaining job still needs something inside the unresolved group. In a finite directed graph, following those dependencies must eventually revisit a node, exposing a cycle.

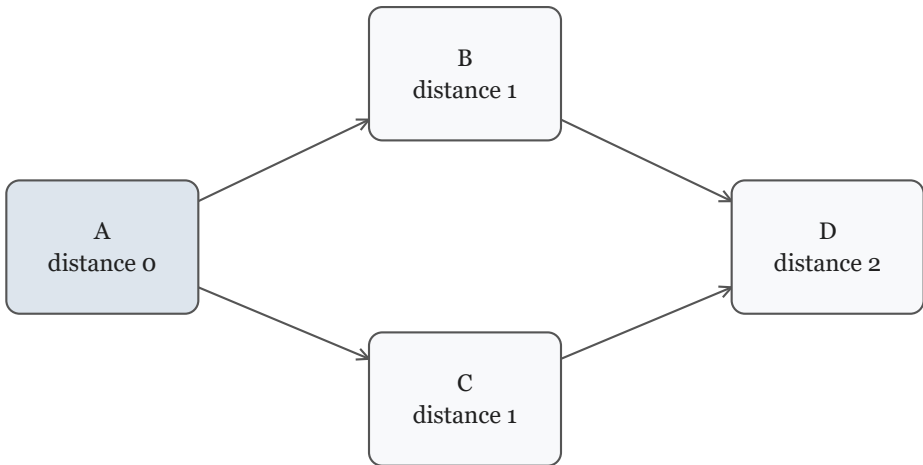
Do not confuse shared descendants with cycles. If A leads to B and C, and both lead to D, D is shared but the graph can still be acyclic. A traversal needs visitation to avoid duplicate work; cycle detection needs

evidence of a path that returns to an active ancestor, or equivalent indegree reasoning. “I saw this node before” alone is not enough to diagnose a cycle in a directed graph.

Worked example: two routes to the same vertex

Consider edges A to B, A to C, B to D, and C to D. Starting at A, find the minimum number of edges to D. Breadth-first search processes the frontier in layers. The queue is not just storage for pending work; its order represents increasing distance from the start.

BFS: DISCOVERY PROCEEDS ONE HOP AT A TIME



Every edge costs one hop. Mark D when it is enqueued.

Every edge costs one hop. B and C both have edges to D, but only the first discovery enqueues D.

Removed from queue	Newly discovered	Queue afterward
A, distance 0	B and C, distance 1	B, C
B, distance 1	D, distance 2	C, D
C, distance 1	None; D already discovered	D
D, distance 2	None	Empty

Mark a vertex when it is enqueued. When C examines its edge to D, D is already discovered even though it has not yet been removed from the queue. Marking only on removal allows duplicate queue entries. Save D's predecessor as B on first discovery; following predecessors backward reconstructs A, B, D. A, C, D is equally short, so a requirement for deterministic paths may also specify neighbor order.

The distance argument assumes every edge has equal cost. If A to B costs ten while A to C and C to B each cost one, the one-edge route is not cheapest. The frontier must then be ordered by accumulated cost rather than by hop layer, as in Dijkstra's algorithm for nonnegative weights.

Worked example: a dependency queue means something else

Interpret the same diamond as build dependencies: A must finish before B and C; both B and C must finish before D. The initial indegrees are A:0, B:1, C:1, D:2. The ready queue starts with A. Completing A makes B and C ready. Completing B reduces D to one unfinished prerequisite; it does not release D. Completing C reduces D to zero and releases it.

The queue now means all prerequisites are satisfied, not minimum distance. This is why naming the container is only half a solution. Its membership rule carries the correctness argument.

Add a dependency D to A. Every vertex now has a positive indegree, so no work is initially ready. The process finishes with fewer completed vertices than exist in the graph, proving the dependency graph contains

a cycle. In a graph with an independent vertex E, E could finish while the cyclic component remains stuck. An empty final queue by itself therefore says nothing; compare completed count with total count.

Across both uses, store every declared vertex, including isolated ones. Building the vertex set only from outgoing edges can silently lose a leaf or a standalone job.

Putting the program together: a prerequisite planner

The function takes a number of jobs and pairs of prerequisite and dependent IDs. It returns a valid execution order and a success flag. IDs lie in a specified range; invalid endpoints fail validation. A cycle means no complete order exists. Independent jobs may appear in any order unless the contract requests deterministic tie-breaking.

Create an adjacency slice for outgoing dependency edges and an integer indegree slice. For each validated pair, append the dependent to the prerequisite's neighbors and increment the dependent's indegree. The counts and edge list must treat duplicate pairs consistently. Initialize a queue with every zero-indegree job, including isolated jobs.

Use a head index to consume that queue. Each removed job becomes the next output. For every dependent, decrement its outstanding count; when the count reaches zero, append that dependent to the queue. No other condition makes a job ready. After the queue is exhausted, compare output size with the total number of jobs. Equal means success; smaller means a cycle prevented completion.

For A before C and B before C, initial counts are zero for A and B and two for C. Finishing A drops C to one. Finishing B drops C to zero and makes it ready. A, B, C is valid, as is B, A, C. Add an isolated D and it must still appear somewhere. Add C before A and a dependency cycle now blocks at least A and C.

Validation, graph construction, readiness processing, and final completeness checking are useful conceptual stages. Their combined

work is linear in vertices plus edges. If the smallest available job must always be chosen, replace the ready queue with a min heap, accepting logarithmic ready-set operations. This changes tie policy rather than dependency meaning.

Go example: record discovery when enqueueing

This complete BFS returns minimum hop counts from a start vertex. Missing adjacency entries mean no outgoing edges. Vertex IDs are strings, and edges have equal cost. The slice is a queue with a head index, avoiding removal from its front.

```
func HopDistances(edges map[string][]string,
    start string) map[string]int {
    distance := map[string]int{start: 0}
    queue := []string{start}
    for head := 0; head < len(queue); head++ {
        from := queue[head]
        for _, to := range edges[from] {
            if _, seen := distance[to]; seen {
                continue
            }
            distance[to] = distance[from] + 1
            queue = append(queue, to)
        }
    }
    return distance
}
```

In the diamond graph, B records D's distance as two before C is processed. C then sees D already in the map and does not enqueue it again. The map doubles as the visited set and the result, so there is no second membership structure to keep synchronized. A cycle back to A also stops because A was registered at initialization.

An unreachable vertex is absent from the result; distance zero belongs to the start and does not mean unreachable. The caller must use the same comma-ok lookup introduced in chapter one. Work is linear in reachable vertices and their outgoing edges. The head-index queue retains its backing array until this traversal finishes; its maximum retained storage is linear in discovered vertices.

Chapter 5 - Stacks greedy choices and dynamic programming

When a problem does not reduce to lookup or ordering, ask what choices remain and whether different paths reach the same remaining problem.

Stacks remember unfinished work

Balanced delimiters are the simplest example. Push opening brackets. A closing bracket must match the most recent unfinished opening bracket, so inspect the top. If it does not match, reject immediately. Finish with an empty stack. Counts alone fail on “open square, open round, close square, close round” because totals ignore nesting order.

A monotonic stack is useful for “next greater” questions. Suppose daily loads are seventy, seventy three, seventy one, seventy four. Store indices whose next greater load is unresolved, keeping their values decreasing from bottom to top. When seventy four arrives, it resolves seventy one and seventy three; pop each and compute its distance. Each index is pushed once and popped once, so total work is linear even with a nested loop.

The proof is about discarded candidates. A value popped by a later greater value no longer needs to wait: this is the first greater value encountered since its insertion. For “next greater or equal,” change the equality rule. For a sliding-window maximum, use a monotonic deque because old indices must also expire from the front. A regular stack cannot efficiently remove those oldest elements.

Greedy requires a reason that local choices are safe

Choose the maximum number of nonoverlapping screenings. Sorting by earliest finish and taking the next compatible screening works: replace the first screening in any optimal schedule with the earliest-finishing one. It cannot finish later, so all later choices remain feasible. Repeat the exchange argument on the remaining schedule.

Sorting by earliest start does not work: a long early interval may exclude many shorter intervals. If screenings have different values and you want maximum total value, earliest finish is no longer enough. The objective changed, so the proof no longer applies. That variant naturally introduces dynamic programming.

Backtracking explores choices deliberately

Generate playlists of exactly three distinct titles from a small set, subject to a duration budget. A state includes chosen titles, next available index, and remaining budget. Choose a title, recurse, then undo the choice. Choosing indices in increasing order avoids generating permutations when order does not matter. If order matters, that restriction would wrongly remove valid playlists.

Prune only when a branch cannot recover. Exceeding a duration budget is safe to prune if all future durations are nonnegative. It is not safe if negative adjustments exist. Copy the current slice when saving a completed answer; otherwise later mutations may overwrite earlier answers through the same backing array. The search can remain exponential, and the output itself may be exponential. Do not hide that cost behind “recursion.”

Failure case. Two paths can reuse a suffix answer only when their remaining constraints and requested output agree.

For counting or best achievable future score, earlier history may be irrelevant once the state summarizes every future constraint. For enumerating full playlists, you can reuse suffix results carefully, but

outputs must still include each distinct prefix. If a genre restriction depends on earlier choices, index and budget are not a sufficient state.

Dynamic programming is reused subproblem reasoning

Consider choosing nonadjacent episode promotions to maximize total value. At index i , either skip it and solve from i plus one, or take it and solve from i plus two. Define best of i as the best total achievable from index i onward. The recurrence is the larger of those two choices. Beyond the end, the best value is zero. If selecting nothing is allowed, negative values naturally get skipped.

For values four, one, one, four, the answer is eight from the first and last positions. Recursive branching repeats the same suffix problems. Memoization reduces the number of solved states to n . A bottom-up pass from right to left uses only the next two answers, giving linear time and constant auxiliary space. If you must return the selected indices, store decisions or a full table and reconstruct. Reference: `MaxNonAdjacent`.

Before writing a DP table, answer four questions in sentences: what does one state mean; what choices leave it; what smaller states do those choices require; and what are the base cases? Then count states times work per state. For a duration budget B and n titles, a table indexed by item and budget may cost $O(nB)$, which is pseudopolynomial because B 's numeric magnitude matters.

How to discover a state instead of guessing a recurrence

Start with a choice you can explain in everyday language. Suppose promotional slots lie in a row and adjacent slots cannot both be selected. At the first slot, you either take it or skip it. Taking it removes the next slot from consideration. Skipping it leaves the next slot available. Both branches lead to smaller problems of the same kind. You have found a recurrence by describing valid choices, not by looking for a formula.

Now ask what information a smaller problem needs. If all earlier constraints have already been resolved, the suffix starting at a position is enough. It does not need to remember the exact sequence of earlier decisions. Two branches reaching that same suffix can reuse the same best result. If there is also a remaining money budget, position alone is no longer enough; the state needs budget too. A dynamic-programming state is a summary of all past information that can still affect the future.

There are two different opportunities to avoid work. A greedy proof shows that one choice can always replace another without harming an optimal answer, so you never explore the rejected alternative. Dynamic programming explores the meaningful alternatives but shares repeated subproblems. Backtracking explores a decision tree and may prune branches that cannot lead to valid answers. These techniques can coexist, but they eliminate work for different reasons.

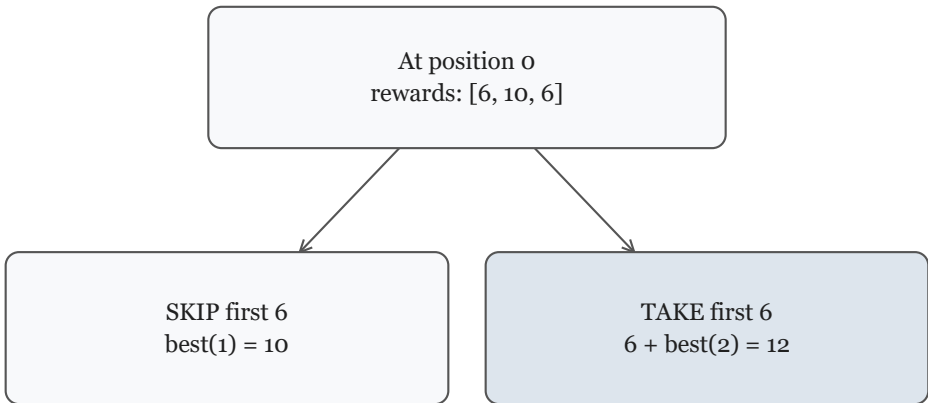
The safest way to test a proposed state is to try to make it lie. Can two histories produce the same state but have different legal futures or different optimal future values? If so, the state has forgotten something important. For example, a playlist state containing only remaining duration cannot enforce “do not choose two films from the same director” unless it also remembers selected directors or uses another representation that preserves the restriction.

Similarly, test a greedy choice by trying to defeat it with a tiny example. Choosing the highest-value promotion first sounds plausible. With values six, ten, six in three adjacent positions, taking the middle ten blocks both sixes, while taking the ends gives twelve. The counterexample shows that this greedy rule lacks the exchange property you would need. It does not mean greedy algorithms are unreliable; it means each greedy rule needs its own proof.

Worked example: discover the state from the choices

Three advertising slots offer rewards 6, 10, and 6. Adjacent slots cannot both be chosen. Choosing the largest reward first gives ten, but choosing both ends gives twelve. The failed greedy choice suggests comparing complete alternatives rather than committing to the locally largest value.

COMPARE COMPLETE ALTERNATIVES, NOT LOCAL REWARDS



Choose 12: positions 0 and 2. Shading marks the winning branch.

At each position, either skip it or take it and jump past its neighbor. The two branches solve smaller suffixes.

Let best at position i mean the largest reward obtainable from position i onward. Skip gives best at i plus one. Take gives the current reward plus best at i plus two. Keep the larger. Past the end, the reward is zero. This definition includes the assumption that choosing nothing is allowed.

Position, evaluated backward	Skip	Take	Best
Last 6	0	6 + 0	6
Middle 10	6	10 + 0	10
First 6	10	6 + 6	12

The suffix state is sufficient because the earlier choices impose no further condition once entry into that suffix is legal. If the problem also limits the number of chosen slots, position alone is insufficient: two visits to the same position with different remaining allowances can have different answers. Add the allowance to the state. Memoization works only when the state actually identifies an equivalent remaining problem.

To recover the selected slots, store which branch won or retain the table and compare alternatives again while walking forward. Computing only the best number can use two rolling values; reconstructing a selection needs additional information. The requested output changes the representation.

Worked example: a stack postpones decisions

For each value in 2, 1, 3, find the next strictly greater value to its right. A stack holds indices still waiting for an answer. After reading two, index zero waits. After reading one, both zero and one wait, with the smaller value at the top. Reading three answers both unresolved positions: first one, then two. Index two then waits until the input ends and receives no answer.

New value	Values still waiting	Answers established
2	2	None
1	2, 1	None
3	3	Next greater for 1 and 2 is 3

Store indices rather than just values so answers can be written to the correct positions, including duplicates. For a strictly greater query, an equal arrival does not resolve an equal waiting value. That single word in the contract changes the comparison.

Every index is pushed once and popped at most once. Several pops during the final arrival do not make the complete scan quadratic. This is the same aggregate accounting used for sliding-window removals, applied to a different invariant: the stack contains precisely the unresolved positions in monotonic value order.

Putting the program together: choosing nonadjacent promotions

The function takes a slice of signed rewards and returns the greatest total from nonadjacent positions. Empty selection is allowed. The result uses a numeric type large enough for allowed sums. An empty input therefore returns zero. We first solve for the value alone; returning selected positions is a separate extension.

Define *best-from-i* as the best achievable total using positions *i* onward. At *i*, skipping gives *best-from-next*. Taking gives the current reward plus *best-from-two-ahead*. Choose the larger. Beyond the input, the contribution is zero. This description is already executable recursive pseudocode, but a naive recursion repeats many identical suffix problems.

A bottom-up implementation walks from the last position toward the first. Keep two fields of working state: the best result for the next suffix and for the suffix two positions ahead. Compute the new best using their old values, then shift the working state so it refers to the correct suffixes for the next iteration. Updating a variable too early can accidentally replace a needed old result.

For four, one, one, four, begin beyond the end with zero and zero. The final four produces best four. The preceding one produces best four. The next one produces best five. The first four produces best eight by

combining itself with the best suffix beginning at the third position. The returned eight corresponds to selecting the two ends.

The loop uses linear time and constant auxiliary storage. To return the chosen positions, keep the table of suffix answers or equivalent decisions, then walk forward: compare taking with skipping under a defined tie policy. Taking advances two positions; skipping advances one. This reconstruction requires information discarded by the two-number optimization, so the extra output requirement affects space.

Go example: keep only the two suffix answers needed

This complete function computes the maximum reward when neighboring positions cannot both be selected. Selecting nothing is allowed. Rewards and their sums must fit int64. The code returns the best number, not the chosen positions.

```
func BestNonAdjacent(reward []int64) int64 {
    var next, afterNext int64
    for i := len(reward) - 1; i >= 0; i-- {
        take := reward[i] + afterNext
        current := next // Skip position i.
        if take > current {
            current = take
        }
        afterNext = next
        next = current
    }
    return next
}
```

At the start of an iteration, `next` is the best suffix beginning at $i+1$ and `afterNext` is the best suffix beginning at $i+2$. For 6, 10, 6, the successive best values are six, ten, and twelve. Assigning `afterNext = next` before overwriting `next` preserves the old suffix answer. Reversing those assignments would overwrite information still needed.

Empty input returns zero. All-negative input also returns zero under the stated contract. If choosing at least one element is mandatory, the initial zero states and recurrence need reconsideration. Time is linear and extra space constant. Recovering the selected indices would require retaining decisions or recomputing them; the two rolling values alone deliberately discard that history.

Chapter 6 - LRU and weighted caches

A cache exercise is about coordinating identity, order, and capacity without letting those views disagree.

Begin with a contract small enough to implement

Our first cache stores string keys and integer values. Get returns a value and a found flag. Put inserts or overwrites. Successful Get and Put both make the entry most recently used. The capacity is the maximum number of entries. Zero capacity stores nothing. A missing Get does not change order. Expiration and concurrency are separate follow-ups.

LRU means least recently used. It is not least frequently used. A title accessed a thousand times yesterday can be evicted before one accessed once just now. This policy is a useful heuristic for temporal locality, not a guarantee that the most popular items survive. A one-time scan of many unique items can displace frequently reused entries.

Derive the two structures

A map locates a value by key quickly but does not identify the least recently used key. A list ordered by recency identifies the oldest entry quickly but cannot locate an arbitrary key quickly. Combine them: map each key to a node in a doubly linked list. Keep the most recent node at the front and least recent at the back. Each node has its key, value, previous pointer, and next pointer.

Why doubly linked? Removing a known node requires access to both neighbors. With only a next pointer, you would generally have to search for the predecessor. With both pointers, detach by joining the neighbors, then insert at the front. Sentinel head and tail nodes make empty-list and endpoint operations follow the same pointer rules.

There are three core invariants. Every map entry points to exactly one real list node. Every real list node has exactly one map entry under its key. Neighbor pointers agree, and the real-node count never exceeds capacity when a public operation returns. Express these before coding; they tell you what every helper must preserve.

Following the recency state

With capacity two, Put A then Put B gives recency order B, A, listed from most recent to least recent. Get A detaches A and moves it to the front, giving A, B. Put C inserts C, temporarily giving C, A, B. Capacity is exceeded, so evict B from the tail and delete B from the map. Overwrite A with a new value: update the existing node and promote it, giving A, C. Do not create a second A node.

Failure case. Removing a cache node from the list without deleting its map entry breaks membership consistency.

The map can return an evicted value and may attempt to detach a node that is no longer linked, corrupting the list. The map-to-list correspondence fails immediately at eviction, before any later lookup. Test internal invariants after operations, not only returned values.

Implement the helpers before the policy

Write detach, insert-at-front, and remove. Detach updates only list links. Insert-at-front links an already detached node. Remove detaches and deletes from the map. Get then becomes lookup, promote, return. Put becomes update and promote, or allocate and insert, followed by eviction when necessary. Avoid duplicating pointer manipulation in every branch.

Expected Get and Put time is constant for a count-limited cache, assuming fixed-cost keys and values. Space is linear in capacity, including map entries and list nodes. Go's built-in container/list can be appropriate in ordinary code, but your plan specifically calls for implementing the linked list yourself; the provided reference LRU does that.

Weighted capacity changes the accounting

Now each entry has a weight, such as payload bytes. Maintain total weight as an invariant equal to the sum of resident entry weights. On overwrite subtract the old weight and add the new weight. Then evict from the back repeatedly while over budget. One Put can evict many items, so worst-case time is linear in the number of entries removed, not constant. Across an operation sequence, each inserted node can be evicted only after being inserted, which helps amortized analysis.

Choose an oversized-item policy explicitly. In this course, reject a weight greater than the entire budget and leave an existing value under that key unchanged. Check this before mutating. Another reasonable contract removes the old entry but refuses the new one; it must be stated and tested. Negative weights are invalid. Zero-weight entries can defeat a byte-only entry bound, so either disallow them, charge overhead, or use a second count limit.

Suppose budget is ten. Put A weight four, B weight four; Get A; Put C weight five. Before eviction, weight is thirteen and order is C, A, B. Evict B to reach nine. If C instead weighs eleven, reject it before touching A or B. A byte budget for values alone does not bound actual process memory: maps, nodes, allocator overhead, and stale heap records also consume space.

Build the cache from the two questions it must answer

Before imagining pointers, imagine two separate conversations with the cache. A caller asks, “Do you have movie A?” Capacity management asks, “Which resident movie has gone longest without being used?” The first question is about identity; the second is about order. A map answers identity but has no meaningful recency order. A recency list answers oldest and newest but cannot immediately find an arbitrary movie. The combined solution is a small example of maintaining multiple indexes over the same objects.

Now imagine a node as a card containing a key and value, with a connection to the card before it and the card after it. The map does not need another copy of the value; it can identify the card directly. When the card moves, its identity remains the same, so the map continues to point to it. Moving a value inside a slice, by contrast, can change positional indexes. A stable node reference lets identity and position vary independently.

Detaching a card means making its two neighbors point to each other. Inserting it at the front means placing it between the head marker and the previous first card. You do not search through the list during either operation because the map already found the card and the card knows its neighbors. That is the actual reason promotion is constant time. A linked list without fast key lookup would still require a search; a key map without backward links would not immediately provide the predecessor needed for removal.

Sentinel nodes are deliberately empty boundary markers. An empty list consists of head connected to tail. The first insertion uses the same “insert between two neighbors” rule as later insertions. They reduce exceptional pointer cases; they do not count toward capacity or belong in the key map. A useful invariant is: every real card is reachable exactly once between the markers and is named exactly once by the map.

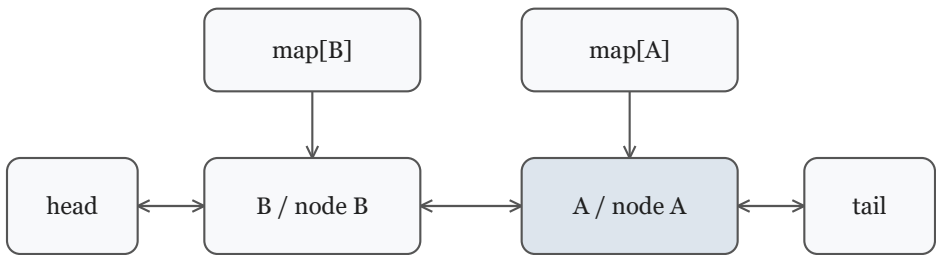
Weighted capacity adds another view: the total charge for all resident cards. That number is a cached summary of the list contents. If you overwrite an entry, you must remove the old charge before adding the new one. If you evict several entries, subtract each exactly once. This is not merely arithmetic bookkeeping; it is an invariant connecting state representations. The same style of reasoning later governs streaming sums when events expire.

Worked example: the map finds the node; the list moves it

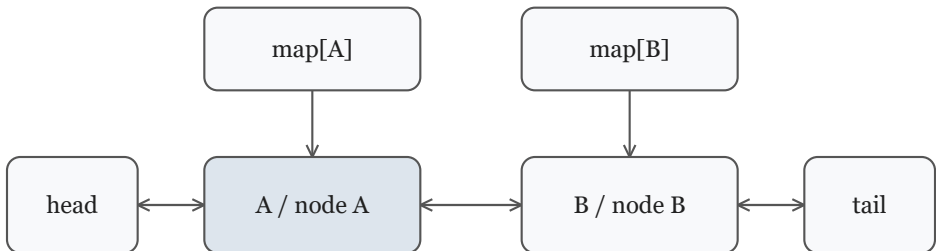
Take a capacity-two cache with recency order B, A. A successful Get for A must find A and move it in front of B. A map supplies a reference to A's existing node. The list supplies its neighbors. Moving the node does not change the map reference, because the object is the same even though its position changes.

LRU PROMOTION: SAME OBJECTS, DIFFERENT NEIGHBORS

BEFORE Get(A): B is most recent



AFTER Get(A): A is most recent



Double arrows = next/prev links. Sentinels are not in the map.

The key index points to stable nodes. A hit on A changes list order from B, A to A, B without replacing the A node.

Operation	Most recent to least recent	Map keys	Result
Put A=4	A	A	One resident
Put B=7	B, A	A, B	Full
Get A	A, B	A, B	Return 4
Put C=9	C, A	A, C	Evict B
Put A=0	A, C	A, C	Update existing node

The zero in the final row is a legitimate cached value. A later Get A returns zero and found true; a Get B returns found false. The map's presence information must not be replaced by a value sentinel.

During a move, think of the pointers as two separate local edits. First join A's old predecessor and successor. Then insert A between the head sentinel and the old first real node, repairing both directions. The helpers can temporarily leave A detached, but a completed public operation must restore one map entry per real list node and matching forward and backward links. Testing only returned values can miss an internal violation until much later.

Worked example: an overwrite can trigger several evictions

Use a byte budget of ten and recency order C, B, A with weights two, three, and four. Total weight is nine. Overwrite A with weight eight. Rejecting an item larger than the entire budget does not apply: eight is admissible. Subtract A's old weight and add its new weight, bringing the total to thirteen; promote A to produce A, C, B.

Evict B, the least recent entry, subtracting three. Total becomes ten, so C survives. If A's new weight were nine, total after the overwrite would be fourteen. Removing B would leave eleven, requiring C to leave as well;

only A would remain at weight nine. A single overwrite can therefore have work proportional to several victims.

If A's proposed weight were eleven, the chosen policy rejects the overwrite before any mutation. The old A remains at weight four, the order stays C, B, A, and total remains nine. This failure case explains why input validation belongs before eviction and why a policy must specify what happens to an existing value when its replacement cannot fit.

The recurring design pattern is a shared identity with multiple maintained views. The map, list, and total-weight counter describe the same residents. Every mutating branch must update all affected views together.

Putting the program together: an LRU cache step by step

The public methods are Get and Put, with a nonnegative entry capacity. Get returns a value and found flag; a hit updates recency. Put overwrites or inserts and also updates recency. Zero capacity retains nothing. The cache struct contains capacity, a map from key to node, and head and tail sentinel nodes. Each real node contains key, value, previous node, and next node.

Define three helpers. Detach joins a node's two neighbors and clears its own links. InsertFront places a detached node between head and the current first node, repairing both directions. Remove calls Detach and deletes the key from the map. Helpers have preconditions: do not detach a sentinel or a node that is not linked, and do not insert a node that is still linked elsewhere.

Get looks up the key. A miss returns missing without mutation. A hit detaches the node, inserts it at the front, and returns its value. Put first handles zero capacity. If the key exists, update that node's value, promote it, and return. Otherwise allocate a node, register it in the map, and insert it at the front. If capacity is exceeded, remove the node immediately before tail.

For capacity two, Put A then B produces B, A. Get A produces A, B. Put C produces C, A after evicting B. Overwrite A with value zero produces A, C; Get A must return zero with found true. This last observation tests the absence representation independently of the ordering.

The public methods preserve one-to-one map/list membership and valid neighboring links. Every operation has expected constant work under the count capacity. If concurrency is added, protect the entire public transition with one mutex first; Get requires the exclusive lock because promotion mutates links. Do not separately lock each helper in a way that leaves inconsistent intermediate states visible or recursively reacquires the same mutex.

Go example: move a known node with local pointer edits

These are complete list helpers and a cache Get method, not the full cache. The constructor must allocate an empty key map and connect head.next to tail and tail.prev to head. Head and tail are sentinels outside the key map. Put and eviction follow the program design above. No concurrent callers are assumed.

```
type cacheNode struct {
    key string
    value int
    prev, next *cacheNode
}

type ReadCache struct {
    byKey map[string]*cacheNode
    head, tail *cacheNode
}

func detach(n *cacheNode) {
    n.prev.next = n.next
    n.next.prev = n.prev
    n.prev, n.next = nil, nil
}

func insertFront(head, n *cacheNode) {
    first := head.next
    n.prev, n.next = head, first
    head.next = n
    first.prev = n
}
```

Detach assumes a linked real node. InsertFront assumes a detached node and a valid sentinel head. Those preconditions let each helper avoid unnecessary boundary branches. Neither helper changes the key map: promotion moves the same object rather than replacing its identity.

```
func (c *ReadCache) Get(key string) (int, bool) {
    n, found := c.byKey[key]
    if !found {
        return 0, false
    }
    detach(n)
    insertFront(c.head, n)
    return n.value, true
}
```

With B followed by A, Get A joins B to tail, then inserts A between head and B. The map continues to point to the same A node. Getting the already-first node is also safe: detach and reinsert restore its position. A missing key takes no pointer path. Count the assignments rather than the list length to see why a hit does constant list work. If concurrency is added, the lock must cover the entire Get transition, including both helpers.

Chapter 7 - Expiration clocks and concurrency

Time introduces a second order. Concurrency makes a sequence of correct individual steps unsafe unless they form one protected operation.

Expiration is a behavioral rule first

For our TTL cache, Put at time t with positive lifetime d stores an absolute deadline $t + d$. Get at time now returns a value only when now is strictly before the deadline. At the deadline it is expired. A nonpositive lifetime removes the key immediately. Updating resets the lifetime. A Get does not extend it. These choices distinguish expire-after-write from expire-after-access.

Use an injected clock so tests control now directly. The learning code uses `int64` ticks in one agreed unit and assumes monotone time and no arithmetic overflow. In a real Go process, a clock returning `time.Time` can preserve monotonic elapsed-time behavior; persisted timestamps need different care. The important point is to make the source and meaning of time explicit, and never rely on real sleeping in unit tests. [Go time documentation](#).

A map with per-entry deadlines gives expected constant-time lookup. On Get, delete an expired entry and return missing. This is lazy expiration: it prevents stale reads, but an entry never read again may remain allocated indefinitely. Correct visibility and prompt memory reclamation are different requirements. Start with the simple version, then explain the gap honestly.

Choose cleanup by the required order

A periodic scan checks every entry in linear time per sweep. It is simple and may be sufficient. A min heap of deadlines makes the next expiry easy to find. Repeatedly pop while the smallest deadline is at or before now. Removing r records costs $O(r \log h)$ where h is heap size. A queue only works if expiry order matches insertion order, such as a uniform TTL with monotone writes. Different TTLs break that assumption.

On each Put, store the current value, deadline, and a new generation number in the map; push deadline, key, generation into the heap. On cleanup, delete the map entry only if its generation still matches the popped record. This prevents an old timer from deleting a newer value. Assign generations from a globally increasing sequence, or otherwise avoid reusing an old generation after a key is deleted and recreated.

Trace A written at time zero, expiring at ten, generation one. At time five, overwrite A to expire at thirty, generation two. At time ten, the old heap record is ready. The map holds generation two, so discard the record without deleting A. The same defense applies to cancelled or rescheduled tasks. Reference: `TTLCache`.

Failure case. Two different writes of the same key can share a deadline, so a deadline alone need not identify a write.

The same key can identify multiple lifetimes. Equal deadlines may be reused, and future changes can make deadline-only identity fragile. A unique generation identifies the particular write, so cleanup can prove it is acting on the current lifetime.

Lazy invalidation simplifies updates but can accumulate stale heap records. Ten million rewrites of one key can produce ten million heap entries despite a one-entry map. Bound this by using an indexed heap with one live record per key, or periodically rebuild from live entries when stale overhead crosses a threshold. Rebuilding with `heap.Init` is linear in live entries, but creates a latency spike. “ $O(\text{number of keys})$ memory” is not true for an unbounded lazy heap.

Combining TTL and LRU

The map still answers identity. The linked list answers least recently used. The deadline heap answers earliest expiration. These are independent orders. An expired node can be at the front of the recency list, so removing only expired nodes at the tail does not clean all expired entries.

Choose whether capacity counts all physically stored entries or only unexpired ones. For “evict expired entries before live ones,” a straightforward Put first drains all due expirations, then updates or inserts, then evicts LRU entries if capacity is exceeded. That Put is not constant time when many items expire together. Get can remain approximately constant time by checking only its own key and leaving global cleanup elsewhere.

Removal must consistently update the map, recency list, and weight counter if present. Heap records may remain as harmless stale versions under lazy invalidation. Reuse a central remove helper rather than implementing three subtly different removal paths for eviction, expiration, and explicit deletion.

In your plan's capacity-three example, assume all actions occur before any deadline: Put A, Put B, Get A, Put C, Put D leaves D, C, A and evicts B. If A expires before D is inserted and cleanup removes expired entries first, A leaves and B can remain. Without specified elapsed time and cleanup policy, the original trace has no unique answer. Noticing that ambiguity is part of the skill.

Thread safety protects the whole invariant

For a first concurrent implementation, put one mutex around each public operation that touches shared state. A successful LRU Get mutates the list, so it needs an exclusive lock. A TTL Get may delete an expired entry, so it is not automatically a read-only operation either. Protecting the map alone while updating pointers outside the lock leaves races and structural corruption.

Do not copy a cache containing a used mutex. Return pointers from constructors and use pointer receivers. Do not try to upgrade an RWMutex read lock into a write lock. A specialized concurrent map would protect only map operations, not the combined map/list/weight invariant. These synchronization rules are grounded in the [Go sync documentation](#) and [memory model](#).

Slow loading introduces another concern. Holding the global cache mutex while calling a metadata service blocks unrelated keys. Instead check the cache, coordinate a per-key in-flight call, release the lock, load, then publish the result and wake waiters. Exactly one leader loads a key; other callers wait on that call's completion. Clear the in-flight record on failure as well as success. Decide whether failures are cached, whether stale values may be served, and how cancellations affect shared work.

Single-flight loading prevents duplicate simultaneous work; it does not enforce TTL or capacity. If a key is invalidated while loading, decide whether the late result may repopulate it. An invalidation generation can prevent that. With sharding, locks can reduce contention, but a globally exact LRU order becomes harder; mention the trade-off rather than claiming sharding preserves every property for free.

Separate freshness, storage, and coordination

Three questions often get bundled together in a cache prompt. Is this value fresh enough to return? Is this entry still taking space? Is another caller already fetching its replacement? TTL answers the first question, cleanup answers the second, and in-flight coordination answers the third. A correct answer to one does not automatically answer the others. An entry can be expired but allocated, and a missing entry can have a fetch already underway.

Use a bookstore analogy carefully. An edition can become outdated at a known time even while its book remains on the shelf. A customer asking for that book must not be handed it as current. Removing it from the shelf can happen during that request or in a separate cleanup pass. Ordering shelves by most recent customer use does not order books by

when their editions expire. That is why recency and deadline often require separate structures.

Now consider an expiry message as a delayed instruction: “Delete A at time ten.” If A is replaced before time ten, the instruction lacks enough identity. It should say, “Delete the particular lifetime of A that I was created for.” A generation number provides that identity. Cleanup looks at the current generation and acts only if it matches. This prevents stale work from affecting newer state. The principle applies to rescheduled tasks, cancelled requests, and late background computations.

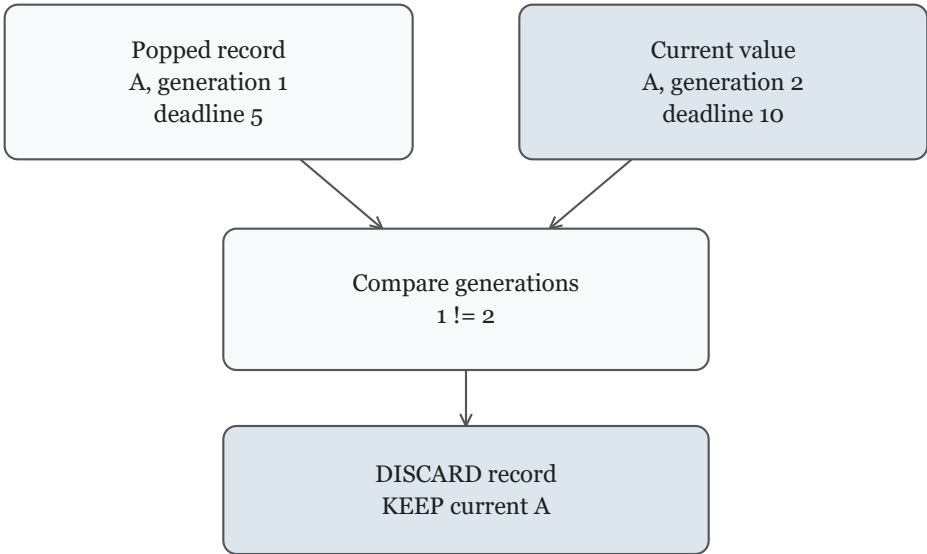
Concurrency creates an analogous problem in the present rather than the future. Two requests can observe the same apparently available capacity before either records its own change. Each individual read and write may be safe, but their combined decision is not atomic. You need a single protected transition from the old valid state to the new valid state. For a cache, that transition can involve map, list, counters, and deadlines together.

Think about the exact moment at which an operation takes effect. In a simple locked cache, it occurs within the critical section while other callers cannot interleave. If you release the lock halfway through promotion, another operation can see a state that is neither the old order nor the new order. Locks are therefore protecting a semantic invariant, not merely preventing a runtime complaint about concurrent maps.

Worked example: an old deadline is not permission to delete

At time zero, store A with deadline five and generation one. At time three, overwrite A with a new value, deadline ten, and generation two. The current map holds only generation two, while a lazy deadline heap can still contain both records. This is intentional: removing arbitrary old heap records immediately would need more indexing.

STALE EXPIRY RECORDS CANNOT DELETE A REFRESH



The due record refers to A generation one. The map contains generation two, so cleanup ignores the old record.

Time	Action	Current A	Heap effect
0	Initial write	Generation 1, due 5	Push (5, A, 1)
3	Refresh	Generation 2, due 10	Push (10, A, 2)
5	Cleanup	Generation 2 survives	Pop and ignore generation 1
10	Get A	Missing	Remove current entry

At time five, checking only the key would wrongly delete the replacement. Comparing the deadline happens to distinguish these two records, but two writes can intentionally share a deadline. The generation identifies the write even then. Deleting A and later reinserting it must also assign a fresh generation rather than restarting a per-entry counter at one.

After the Get at time ten, a heap record may remain for a now-absent key. Later cleanup ignores it. The essential rule is that stale records cannot act on newer state. Space analysis must include those records; repeatedly refreshing one resident key can grow the heap far beyond the resident count unless cleanup or rebuilding bounds it.

Worked example: individually safe operations can form an unsafe sequence

Suppose a deduplication wrapper has a thread-safe Get and a thread-safe Put. Worker one calls Get X and sees missing. Worker two calls Get X and sees missing. Both process the event, then both call Put X. No data race is necessary for this logical failure. Each method can be protected correctly while the combined decision is not atomic.

Step	Worker one	Worker two
1	Looks up X: missing	Waiting
2	Paused	Looks up X: missing
3	Processes X	Processes X
4	Stores X	Stores X

An atomic claim operation combines checking absence and recording ownership under one lock. Only the worker that creates the claim proceeds. That fixes competing admission, but it introduces a new failure question: if the winner crashes after claiming, should another worker retry? A lease can allow a later retry, while an idempotent effect can make repeated execution tolerable. Neither follows automatically from a mutex.

For a cache miss that triggers a slow load, the corresponding pattern is per-key in-flight work. Under a short lock, create or join the work record. Release the lock during the network call. Publish success or failure and release all waiters afterward. The design has to cover the failure branch as carefully as the successful return.

Putting the program together: TTL storage with safe cleanup

Expose `Put(key, value, lifetime)`, `Get(key)`, and `Cleanup`. A clock is supplied to the constructor. The store owns a map of current entries, a min heap of expiry records, and a generation sequence. Each current entry contains value, deadline, and generation. Each heap record contains key, deadline, and generation. Time units are consistent, deadlines fit the numeric type, and generation identities are not reused during relevant lifetimes.

`Put` with a nonpositive lifetime removes the current map entry. Otherwise it reads the clock, computes the deadline, allocates a fresh generation, stores the current entry, and pushes an expiry record. An older record for that key can remain in the heap. `Get` returns missing if absent. If the clock is at or past the current deadline, it deletes the map entry and returns missing. Otherwise it returns the value. `Get` does not drain the whole heap.

`Cleanup` reads now once for its cutoff. While the heap root is due, pop it. Look up its key in the current map. If absent, do nothing further. If present with a different generation, discard this stale record. Only a matching current generation may be deleted. A returned removal count can count current entries actually removed, rather than all stale heap records examined.

Write `A` at zero with deadline ten. At five, overwrite it with deadline thirty and a new generation. `Cleanup` at ten pops the old record but leaves the map unchanged. `Cleanup` at thirty removes the new entry. Deleting and recreating `A` between those calls is safe only if recreation does not reuse the old generation.

`Put` costs logarithmic time in heap size; `Get` is expected constant time; `Cleanup` costs according to the number of records popped. Heap size includes stale records, so repeated overwrites can grow memory. An indexed heap or periodic rebuilding addresses that extension. Adding a single mutex around public operations protects state, provided the injected clock is safe and does not reenter this cache.

Go example: validate a due record against current identity

This helper applies one expiry record after a deadline heap yields it. For this standalone TTL map, deadline is in integer ticks and generations are fresh for every write, including delete-and-reinsert. Map access is owned by one goroutine or one external lock. A combined LRU cache must call its central removal helper instead of only deleting from the map.

```
type TimedValue struct {
    Value string
    Deadline int64
    Generation uint64
}

type Expiry struct {
    Key string
    Deadline int64
    Generation uint64
}

func ApplyExpiry(current map[string]TimedValue,
    record Expiry, now int64) bool {
    if record.Deadline > now {
        return false
    }
    entry, found := current[record.Key]
    if !found || entry.Generation != record.Generation {
        return false
    }
    delete(current, record.Key)
    return true
}
```

If the popped record is A generation one while the map holds generation two, the comparison prevents deletion. A missing key also returns false. At exactly the deadline, a matching generation is removed. The boolean reports an actual resident removal, not merely processing a heap record.

The helper relies on the invariant that a matching generation identifies a write whose stored deadline equals the record's deadline. Get must still check visibility at read time; a cleanup helper alone cannot promise that a value is fresh when cleanup has not run. Each helper call has expected constant map work; the surrounding heap pop costs logarithmic time in heap records, including stale ones.

Chapter 8 - Rate limiting and rolling windows

The policy determines the data structure. “One hundred per minute” is not precise enough to choose an algorithm.

Define what counts

Our exact limiter allows at most L accepted requests per user in the rolling interval strictly after now minus W and at or before now. Rejected attempts do not count. Requests can share a timestamp. Time is monotone for the limiter and uses integer ticks. Users are independent. At the exact left boundary, an old request no longer counts.

For limit two and window ten, requests at time zero and time one are accepted; time nine is rejected. At time ten, the request at zero leaves the window, so the new request is accepted. The retained accepted timestamps are one and ten. Writing this trace before code settles the off-by-one choice.

Exact sliding log

Store a map from user ID to a queue of accepted timestamps. Before deciding, discard timestamps at or before now minus W . If the remaining length is already L , reject. Otherwise append now and accept. Chronological order makes expired records a prefix, so no search is needed. A slice with a head index avoids shifting every remaining timestamp on every removal.

Each accepted timestamp is appended once and removed once, so work is amortized constant per request. A single request after a long idle period may remove L entries and cost $O(L)$. Memory is $O(U \text{ times } L)$ for U active users, plus bookkeeping for idle users if you never delete them. Rejected attempts must not be appended under this contract, or an attacker could keep extending lockout by sending more requests.

With twenty million users and limit one hundred, even bare eight-byte timestamps could occupy sixteen billion bytes if every queue were full, before map and slice overhead. This is an illustrative worst-case calculation, not a measured Netflix workload. Aggregate timestamps within buckets when approximate precision is acceptable; retain exact timestamps when boundary accuracy is required. Delete empty idle users or expire their state, while ensuring cleanup itself does not create unbounded latency.

Failure case. Out-of-order timestamps can hide an expired item behind a fresh queue head.

An expired timestamp might appear behind a fresh one, so removing only a prefix fails. Use monotone server admission time for a request limiter. Event-time analytics with late arrivals is a different problem and may require ordered storage, lateness bounds, or explicit rejection of old data.

Fixed windows are cheaper but allow boundary bursts

A fixed-window counter stores a bucket number and count per user, resetting when the bucket changes. With limit one hundred, a user can send one hundred requests just before a minute boundary and another hundred just after it. That is valid per fixed minute but violates a strict rolling-minute maximum. Say which guarantee the implementation provides.

A two-bucket sliding estimate weights the previous bucket's count by the fraction still overlapping the current window. It is compact but assumes something about the distribution of requests within a bucket. It cannot be exact without more information. At interview scale, describing the approximation and a burst counterexample is more valuable than asserting it is “basically the same.”

Token buckets express burst capacity and sustained rate

A token bucket has capacity C , refill rate r tokens per tick, current tokens, and last refill time. On each request, add elapsed time times r , capped at C , then move last refill time to now. If at least one token exists, subtract one and accept. A bucket starts full in our contract, allowing an initial burst.

With capacity three and refill one token per second, three requests at time zero pass and a fourth fails. At time one, one token has refilled, so one request passes. Long inactivity never accumulates more than three tokens. Fractional tokens allow smooth refill; integer arithmetic with retained remainder is an alternative when exact units are important.

A token bucket with capacity one hundred and refill ten per second is not “at most one hundred per rolling minute.” It intentionally permits a burst plus continued refill. Over an interval of length T , accepted demand is bounded roughly by capacity plus rate times T , subject to initial state and endpoint conventions. Choose it when bursts are acceptable and long-run throughput must be controlled.

Refill must account for rejected requests too: update the timestamp whenever you materialize elapsed refill, or the same elapsed time can be counted again. Our reference clamps a backward clock reading to the previous time. That is defensive behavior, not permission to use arbitrary client timestamps. At very small scales, floating-point arithmetic can affect boundaries; tests use exactly representable examples and document this limitation.

Concurrency and multiple servers

Two simultaneous requests can both observe one remaining slot and both accept unless prune, check, and append form one atomic operation. A mutex solves this inside a process. Per-user locks or shards can improve concurrency when required, but map creation and lock lifetime must still be coordinated. Token refill, check, and debit likewise need one critical section.

Independent per-server limiters can multiply a user's global allowance. A shared atomic decision, stable ownership of each user's state, or intentionally partitioned quotas can address that, with different availability and utilization trade-offs. A distributed system discussion should name failure policy: if the limiter's state service is unavailable, does the request pass or fail? No single answer is correct for every service.

Start from the guarantee a caller should observe

“One hundred requests per minute” can describe several different products. A billing counter might group requests by calendar minute. An abuse rule might forbid any rolling sixty-second interval from containing more than one hundred accepted requests. A service protection rule might permit a burst after inactivity while enforcing a sustained rate. Those are not small implementation variations; they permit different request sequences. A correct implementation begins with the observable guarantee.

An exact sliding log retains the recent accepted requests because each timestamp can affect a future boundary decision. A fixed-window counter forgets where within the bucket those requests occurred. That lost detail is why it cannot distinguish one hundred requests spread throughout a minute from one hundred clustered at the end. Compression of state usually comes with a question: are the histories being merged truly equivalent for the required queries?

A token bucket summarizes a different history. Its token balance says how much accumulated permission remains. Waiting restores permission at a configured rate, but the cap prevents indefinite accumulation. You can think of the balance as unused capacity, rather than as a count of events inside a precise interval. That interpretation explains why a bucket naturally allows bursts and why it does not enforce an exact rolling-window count with the same parameters.

Atomicity matters at the decision point. Imagine the exact limiter has one slot remaining. Two callers independently read that fact, then both

append an accepted request. Every individual access could use a thread-safe map and still produce an invalid combined result. The protected operation must include removal of obsolete events, the limit check, and recording acceptance. For a bucket it must include refill, checking balance, and debiting a token. The state invariant is what defines the lock boundary.

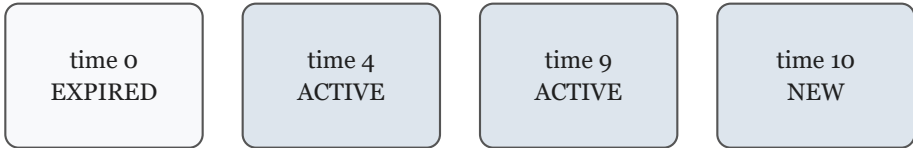
Large user populations add lifecycle questions. A user who disappears does not trigger their own cleanup. If state remains forever, the number of remembered users can grow even while each queue is bounded. Deleting idle state is safe only when reconstructing it gives the same future behavior. An empty exact sliding log can disappear after all counted requests leave the horizon. A token bucket can disappear once it would be fully replenished, assuming a recreated bucket starts full.

Worked example: the exact left boundary matters

Allow at most three accepted requests in any trailing ten seconds. At time t , count accepted timestamps strictly greater than t minus ten and at most t . Requests arrive at 0, 4, 9, 10, and 10. Rejected attempts do not enter the queue under this contract.

EXACT WINDOW: EXPIRE BEFORE TESTING CAPACITY

At now = 10, accepted timestamps must be in (0, 10].



1 Remove 0. 2 Count 4 and 9. 3 Accept the new 10.

A second request at 10 is rejected: three slots are already used.

At time ten the active interval excludes zero and includes ten. Expire the old boundary before testing capacity.

Request time	Expired first	Decision	Queue afterward
0	None	Accept	0
4	None	Accept	0, 4
9	None	Accept	0, 4, 9
10	0	Accept	4, 9, 10
10 again	None	Reject	4, 9, 10

The entry at zero leaves exactly at time ten. Removing timestamps strictly less than the boundary instead of less than or equal to it would incorrectly deny the first request at ten. The method order is therefore observable: read the trusted clock, remove expired entries, compare live count with the limit, then append only if accepted.

The queue argument requires nondecreasing timestamps. If an old timestamp arrives behind a fresh one, stopping cleanup at the first fresh record can leave expired records hidden later. Server admission time

restores the ordered assumption; supporting arbitrary event time requires a different data model.

Worked example: two policies allow different bursts

A fixed-window limit of three per ten-second bucket can accept three requests at 9.9 seconds and three more at 10.1. Each calendar bucket obeys its limit, yet six requests arrive within 0.2 seconds. This is not a coding defect in the counter. It is a weaker guarantee than the exact trailing window.

A token bucket with capacity three and refill rate one token per second expresses another guarantee. It begins full. Three immediate requests consume all tokens. Half a second later, it has half a token, so a unit-cost request is rejected. Another half second later it reaches one token and can admit one request.

Refill must preserve fractional progress if the representation supports fractional time. Discarding the half token on every rejection can prevent recovery under frequent retries. One correct approach computes the new balance from elapsed time, caps it at capacity, and updates both balance and last-refill time even if the request is rejected. An integer-only representation needs an explicit policy for carrying unused fractional time.

The choice follows the requirement: exact recent count needs the timestamp queue; inexpensive calendar-bucket accounting can use a fixed counter; a sustained rate with controlled bursts suits a token bucket. Their similar names do not make their accepted request sequences equivalent.

Putting the program together: exact admission and a token bucket

For the exact limiter, the public operation `Allow(user, now)` returns a boolean. State consists of the limit, window length, and per-user timestamp queues. Each queue has a timestamp slice and a head index. The contract requires nondecreasing admission time and counts only accepted requests in the interval after now minus window through now.

`Allow` finds or creates the user's queue. Advance its head while the oldest retained timestamp is at or before the cutoff. Count the entries from head to the end. If already at the limit, return false without appending. Otherwise append now and return true. Periodically compact the slice when discarded slots become a significant fraction of its contents. The pruning, count check, and append belong to one critical section if callers overlap.

With limit two and window ten, `Allow` at zero and one returns true; at nine false; at ten true. After that last call the active queue contains one and ten. A rejected call must not alter that accepted-event sequence. Another user has independent state. Idle user cleanup is a separate bounded-memory concern; no new request arrives to trigger self-cleanup for a disappeared user.

For a token bucket, the fields are capacity, refill rate, current tokens, and last-accounted time. `Allow` first brings tokens up to now: add elapsed time times rate, capped at capacity, and update the accounted time. If tokens are below one, return false. Otherwise subtract one and return true. Protect that full sequence atomically. Define the clock policy and whether fractional tokens are represented by floating point or exact integer units.

A capacity-three bucket starts with three immediate acceptances and then rejects. After one second at one token per second it accepts one more. Long idle time returns it only to capacity. These tests validate a different guarantee from the exact rolling log. The two structs share an admission API but should not be presented as interchangeable implementations of the same rule.

Go example: expire, count, then admit

This complete single-user limiter uses nondecreasing integer-second timestamps, a positive window, and a nonnegative limit. Timestamps and subtraction must fit int64. Construction supplies those fields; callers are serialized. The method counts accepted requests only.

```
type WindowLimiter struct {
    Window int64
    Limit int
    accepted []int64
}

func (l *WindowLimiter) Allow(now int64) bool {
    cutoff := now - l.Window
    expired := 0
    for expired < len(l.accepted) &&
        l.accepted[expired] <= cutoff {
        expired++
    }
    l.accepted = l.accepted[expired:]
    if len(l.accepted) >= l.Limit {
        return false
    }
    l.accepted = append(l.accepted, now)
    return true
}
```

For Window ten and Limit three, requests 0, 4, 9, 10, 10 produce true, true, true, true, false. At time ten, timestamp zero is removed before capacity is checked. A rejected request is not appended, so retries do not silently change the policy. Limit zero denies every request.

Each accepted timestamp is appended once and discarded once, yielding amortized constant queue work per request. A single request may expire many timestamps. Reslicing retains the backing array; bounded queue length does not mean immediate release of its old allocation. A ring buffer or periodic compaction gives more explicit reuse. Supporting many users adds a map of limiter states and a separate idle-user cleanup policy; it does not change this per-user admission rule.

Chapter 9 - Deduplication

counters versions and scheduling

Many practical tasks reuse the same ideas: key identity, chronological removal, predecessor lookup, and versioned invalidation.

Deduplication needs an identity and a horizon

An event has an event ID, user ID, title ID, duration, and event time. To suppress repeat deliveries, first define identity. The same event ID identifies the same logical event in our exercise; two different IDs with identical payloads are separate events. Store accepted IDs in a set. A repeated ID is rejected, while an unseen ID is recorded and accepted.

With ten-minute retention, use a deadline per ID. Decide whether a duplicate refreshes the deadline. Here it does not: expiration is measured from the first accepted processing time. A repeat just before expiry therefore does not suppress that ID forever. Use server processing time for this exercise, not an untrusted event timestamp. A TTL map with a cleanup heap implements the contract directly.

Now consider a callback that performs the actual side effect. If you mark the ID before the callback and crash, a retry can be suppressed even though the effect never completed. If you mark it afterward and crash in between, the effect can happen twice. A local set is duplicate suppression during its lifetime, not a proof of durable exactly-once effects. Stronger behavior needs an idempotent destination or a transaction connecting the marker and effect. Explain the failure window rather than promising impossible guarantees.

Failure case. Two workers can both observe an absent event before either records it.

Each individual operation may be safe while the combined check-and-record sequence races. It must be one atomic operation

under a shared lock or equivalent coordination. The later side effect still has separate crash and retry semantics.

Hit counters aggregate what the query needs

For a counter of hits in the last sixty seconds with integer-second timestamps, keep timestamp-and-count buckets. Multiple hits in the same second increment one bucket. Remove old buckets and maintain a running total. A queue works for monotone timestamps. A ring of sixty buckets can bound space when the query is always that fixed horizon, but each slot must also store its absolute timestamp to distinguish this minute's second from an older cycle.

At now one hundred fifty, the interval strictly after ninety through one hundred fifty contains events at one hundred, one hundred one, one hundred one, and one hundred fifty: four hits. At now one hundred sixty, the event at one hundred is outside a sixty-second window. State the query's now value explicitly; “last sixty seconds” without a clock cannot determine an answer.

A ring with integer-second buckets is exact for an integer-second event model. For arbitrary subsecond queries it may be approximate at the partial boundary bucket. If queries request many different horizons, a single running total is insufficient; scan retained buckets or use prefix sums over immutable history. Storage must retain at least the largest supported horizon. Out-of-order ingestion also needs an explicit lateness policy.

Time-based values are predecessor queries

Store versions per key ordered by timestamp. For quality at times ten and twenty, a query at fifteen returns the time-ten value. Search for the first timestamp greater than the requested time, then step back. Equal timestamp writes replace in our contract. Earlier writes are rejected so appends preserve order. If historical writes must be accepted, sorted insertion costs linear time in a slice, or you need a structure suited to ordered updates.

Deletion can be a tombstone version: an explicit marker saying the key ceased to exist at that point. Omitting deleted keys from history would make older queries incorrect. Expiring versions is another policy: it may make historical answers unavailable. Always ask whether retention and temporal queries must coexist.

Schedulers are priority queues plus lifecycle state

Schedule takes an ID, a task, and a deadline. RunReady removes tasks whose deadlines are at or before now. A min heap exposes the earliest deadline. Define ordering for equal deadlines, such as insertion sequence. Cancellation marks an ID inactive. Rescheduling creates a new generation and heap record. When an old record surfaces, ignore it unless the active generation still matches.

Do not execute arbitrary callbacks while holding the scheduler's state lock. Collect ready work or claim it under the lock, then release the lock and execute. Decide whether cancellation after a task is claimed can still stop it. If a callback schedules another task due immediately, decide whether it belongs in the current drain or the next call; either choice should avoid an accidental infinite run.

With multiple workers, claim each task atomically. With crashes, a claimed task may never complete. A lease can make abandoned work eligible for retry, which creates possible duplicate execution; idempotent task handlers then matter. That is a discussion extension, not a demand to build a distributed scheduler during a short coding exercise.

Identity and time solve different parts of event processing

Imagine receiving a viewing event twice because a sender retried. The repeated event ID tells you it is the same logical event, but its timestamp tells you where it belongs in time. These are independent attributes. Two distinct events can happen at the same timestamp. The same event can arrive at different processing times. Using time alone as a deduplication key merges unrelated events; using event ID alone does not answer a time-window query.

There are also at least two clocks in the story. Event time says when the viewing action occurred. Processing time says when your service handled a message. A late message may be old by event time but new by processing time. A ten-minute duplicate-suppression horizon measured from first acceptance answers a different question from “include viewing events that occurred during the last ten minutes.” State which one each deadline represents.

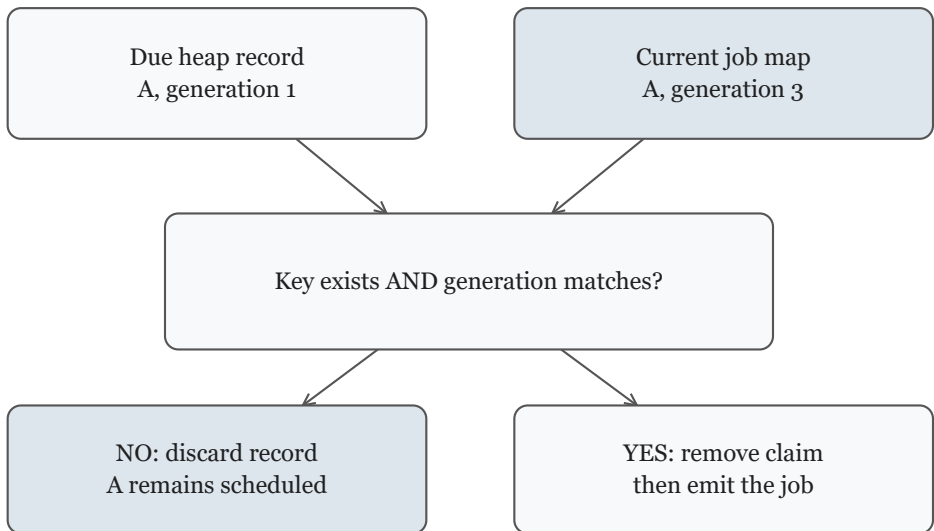
A scheduler reuses the same separation. Task ID identifies what can be cancelled or replaced. Scheduled time establishes priority. Generation identifies which scheduling decision a heap record represents. A map, deadline heap, and generation check therefore appear naturally, just as in TTL cleanup. This is transfer of a reasoning pattern: delayed work must prove it still applies to the current version before mutating state.

Retries reveal the boundary between an in-memory coding problem and a durable service. Suppose you record “processed” before charging a customer, then crash before the charge. A retry is suppressed and the effect is missing. Record it after charging and a crash in between can allow the charge to repeat. The order of two independent actions cannot eliminate both failure windows. You need the destination to recognize repeated identity or a transaction that connects effect and marker. Even if the interview asks only for an in-memory set, accurately naming its guarantee shows good judgment.

Worked example: cancellation invalidates a heap record

A scheduler stores jobs by ID and orders work by due time. Schedule A at time five with generation one, and B at time seven with generation two. Reschedule A to time nine with generation three. The map now says A is due at nine, but the heap can retain the obsolete time-five record.

DRAINING A HEAP IS NOT THE SAME AS RUNNING A JOB



For this record, follow NO. Shading marks the actual outcome.

The heap supplies the next candidate deadline; the current-job map decides whether that candidate is still authorized to run.

Clock or action	Record examined	Outcome
Drain at 5	A, generation 1	Ignore: current generation is 3
Cancel B	Current B removed	Its heap record becomes stale
Drain at 7	B, generation 2	Ignore: no current job
Drain at 9	A, generation 3	Claim and emit A

To claim a valid due job, remove it from the current-job map before returning it to the caller. A second drain then cannot emit the same generation again. For equal deadlines, use an explicit tie rule such as insertion sequence if reproducible order is required. A job ID alone does not distinguish an old schedule from its replacement.

The heap may hold more records than there are active jobs. If thousands of reschedules target a distant deadline, none of those stale records is removed by draining near-term work. Rebuilding from the current map once stale overhead becomes too large is one possible bound. An indexed heap trades more update bookkeeping for immediate removal instead.

Worked example: aggregation forgets what expiration later needs

Suppose title A receives a four-minute event at time two and a six-minute event at time eight. Its total is ten. For a trailing ten-second query at time twelve, the first event is exactly on the excluded left boundary and must leave. The answer is six.

A totals map containing only A:10 cannot tell which contribution expired. Keep the individual active contributions in timestamp order as well as the aggregate. Removing the time-two record subtracts four from A's total. If a title's total becomes zero and durations are positive, remove its aggregate entry.

Time of query	Active interval	Active contributions	A total
9	$(-1, 9]$	A:4 at 2; A:6 at 8	10
12	$(2, 12]$	A:6 at 8	6
18	$(8, 18]$	None	0

Deduplication is a separate history. Forgetting an expired contribution does not necessarily permit forgetting its event ID: a late retry can otherwise be mistaken for new work. Conversely, retaining every ID forever bounds duplicates but not memory. The contract must establish a retry horizon, stable event timestamps, or a durable identity mechanism. The queue answers which contributions are active; the identity store answers which deliveries have already been accounted for.

Putting the program together: a cancellable scheduler

Use `Schedule(id, deadline)`, `Cancel(id)`, and `Drain(now)`, where `Drain` returns ready IDs instead of running callbacks. Scheduling an already-active ID replaces its previous schedule. Cancellation of an unknown ID returns false. Deadlines equal to now are ready. Ties execute in scheduling-sequence order. This deterministic first version makes behavior easy to trace without introducing workers.

Store a map from task ID to its active generation, and a min heap of records containing deadline, scheduling sequence, ID, and generation. A fresh sequence can also serve as a nonreused generation. Heap order compares deadline first and scheduling sequence second. `Schedule` updates the active map and pushes a fresh record. `Cancel` deletes the active map entry and reports whether it existed.

`Drain` repeatedly pops records whose deadlines are due. A missing active ID means cancellation or prior completion, so skip the record. A mismatched generation means replacement, so skip it. A matching generation is current: delete its active map entry and append its ID to

the returned list. Stop when the root is in the future or the heap is empty.

Schedule A for ten, B for five, replace A with three, and cancel B. Drain at four returns A. Drain at twenty returns nothing because the remaining records are cancelled or stale. Schedule two new tasks for the same deadline and their sequence numbers determine stable order. Old records for a reused ID cannot claim a newly scheduled task's lifetime.

Like lazy TTL cleanup, this design can retain stale heap records until their deadlines arrive. State the memory cost. If Drain later runs callbacks, collect or claim work under the lock, release the lock, then execute callbacks. Define whether cancellation after claiming can stop execution. Returning IDs first keeps lifecycle logic separate from arbitrary callback behavior.

Go example: claim a scheduled generation before returning it

This helper handles a record already popped from a due-time heap. It is not the complete scheduler. The map stores each job's current generation, assigned freshly on scheduling or rescheduling. The caller serializes the check-and-delete sequence and invokes external work only after this helper succeeds.


```
type JobRecord struct {
    ID string
    Due int64
    Generation uint64
}

func ClaimJob(current map[string]uint64,
    record JobRecord, now int64) bool {
    if record.Due > now {
        return false
    }
    generation, found := current[record.ID]
    if !found || generation != record.Generation {
        return false
    }
    delete(current, record.ID)
    return true
}
```

After rescheduling A from generation one to three, a due record for one returns false and leaves A scheduled. A record for three at its deadline returns true and removes the claim. Calling again with that same record returns false because the current entry is gone. Cancellation uses the same absence branch.

The caller must peek at the heap deadline before popping: a not-yet-due record must remain queued, even though this defensive helper would reject it. This gives at-most-one claim of a generation within this in-memory state, not exactly-once execution of an external effect. If a worker crashes after the successful claim, durable retries and idempotent effects need a larger lifecycle model.

Chapter 10 - Recommendation and streaming ranking tasks

For a coding interview, turn “recommend movies” into a small, testable pipeline with explicit inputs, filters, scores, and tie rules.

Understand the domain without overbuilding it

Netflix describes recommendations as using member interactions, similar tastes, and title information, with personalization affecting rows and ordering. This gives useful domain context, but it is not an interview specification. [Netflix's recommendation overview](#).

A general recommendation pipeline first retrieves plausible candidates, scores them, and applies final constraints. Content-based retrieval uses item attributes; collaborative methods learn from patterns across users and items. This is a conceptual foundation, not a requirement to train a model in Go. [Google's candidate generation overview](#).

For an interview, ask what is already given. If scores are supplied, your task may be filtering and top k selection. If co-watch histories are given, it may be graph or frequency aggregation. If scores update live, it may be an indexed heap or an ordered-state problem. The title “recommendation engine” does not choose the algorithm.

A precise ranking exercise

Input is a list of candidates with ID, genre, integer score, and eligibility flag, plus a set of watched IDs and k. Exclude ineligible and watched candidates. Deduplicate remaining IDs by retaining their highest score. For equal scores of the same ID, retain the lexicographically smaller genre so metadata is deterministic. Return at most k titles ordered by descending score, then ascending ID. Nonpositive k returns an empty result.

For candidates A score eight, B score nine, A score ten, C score nine, with B watched and k two, the winners are A then C. Deduplication must happen before top k selection, or two copies of A can occupy both slots. Eligibility is applied before deduplication in this contract; if eligibility instead belongs to the canonical catalog record, resolve canonical metadata first. That is a requirements distinction worth raising.

The simple baseline builds a map of unique eligible candidates and sorts its values. For n input records and u unique eligible IDs, expected work is $O(n \text{ plus } u \log u)$, and space is $O(u)$. When k is much smaller than u, use the bounded heap from lesson 3, then sort the k winners. The deduplication map still uses $O(u)$ space even though the heap uses only $O(k)$. Do not claim the whole pipeline uses $O(k)$ memory.

Failure case. Selecting top k before applying a genre restriction can discard needed alternatives.

No. The original top k may all belong to one genre, while the best allowed candidates from other genres lie outside it. Apply the constraint during selection over sufficient candidates, or select each genre's best first under the one-per-genre contract.

Constraints change what top k means

For a single disjoint genre per movie and a cap of one per genre, keeping the best movie of each genre and then selecting the best k genre winners maximizes total score. For a cap greater than one, sorted greedy acceptance respects the per-genre quota. If movies have multiple overlapping tags and several simultaneous quotas, a simple greedy strategy need not be globally optimal. State whether you are maximizing score exactly or applying a reasonable reranking heuristic.

Another common follow-up asks to interleave two ranked lists while removing duplicates. Two pointers with a seen set can preserve a chosen alternation policy. This does not necessarily preserve global score order. Define whether fairness between sources or highest overall scores has priority. If each source is sorted and global score order is required, a heap across source heads gives a k-way merge.

Small collaborative scoring without machine learning

Suppose a target user watched A and B. Neighbor one watched A, C, D; neighbor two watched B, C; neighbor three watched E. Define similarity as number of shared watched titles. Neighbor one and two each have weight one; neighbor three has zero. Add a neighbor's weight to each of their unwatched titles. C receives two and D one. Recommend C before D. Use sets per history so repeat viewing does not accidentally count as multiple distinct overlaps.

This is an original toy scoring rule. It illustrates joins and aggregation, not Netflix's actual ranking formula. An inverted index from title to users can avoid scanning every user to find overlaps, although extremely popular titles can still generate many candidates. Define cold-start behavior when no overlap exists, such as a supplied popularity list filtered by eligibility.

Streaming statistics add reversibility

For average watch duration per video, store sum and count, then divide only when count is positive. For a one-hour window, retain events in a time queue and subtract each expired event's duration and count from its video's aggregate. A running mean by itself does not retain enough information to remove arbitrary old contributions easily. If ordering is not monotone, a FIFO expiration queue is no longer sufficient.

Dynamic top k is harder than one-time top k. Counts can fall when events expire. A size-k heap of only current winners may have forgotten the best replacement outsider. Keep full aggregates and rebuild on query, or maintain an indexed structure over all candidates; discuss the update-versus-query trade-off. An approximate heavy-hitter sketch is a different contract with error and ranking uncertainty.

Make the recommendation problem small enough to reason about

A recommendation prompt can sound enormous because the real product contains retrieval models, ranking models, experiments, data pipelines, and policy rules. A coding exercise usually exposes only a small piece of that system. Your first task is to identify the piece. Are candidate scores already supplied? Are you deriving a toy score from viewing histories? Are you maintaining changing scores over time? These versions require different programs even though they all return movie IDs.

Separate the stages in words. Eligibility decides whether a candidate is allowed at all. Identity resolution ensures one logical title is not counted twice. Scoring assigns a comparable value under an agreed formula. Selection chooses a limited set. Ordering determines the sequence in which the chosen set is returned. These stages can interact: discarding duplicates after selecting winners may leave too few distinct results, and filtering only after selection can hide valid candidates just below the original cutoff.

Think of a supplied score as an input contract, not an explanation of user enjoyment. Your coding task might require ranking by that number without evaluating whether the number is a good model. A toy collaborative rule can be educational, but it should be described honestly as a rule we define for the exercise. It is not evidence of Netflix's actual model architecture or training objective.

Determinism is part of the API. Two candidates with equal scores need an agreed order if callers or tests compare results. A map's iteration order should not accidentally decide which result wins. Ties become particularly subtle at a heap boundary because the heap keeps the worst retained winner at its root. You must reverse both the score direction and the tie direction consistently.

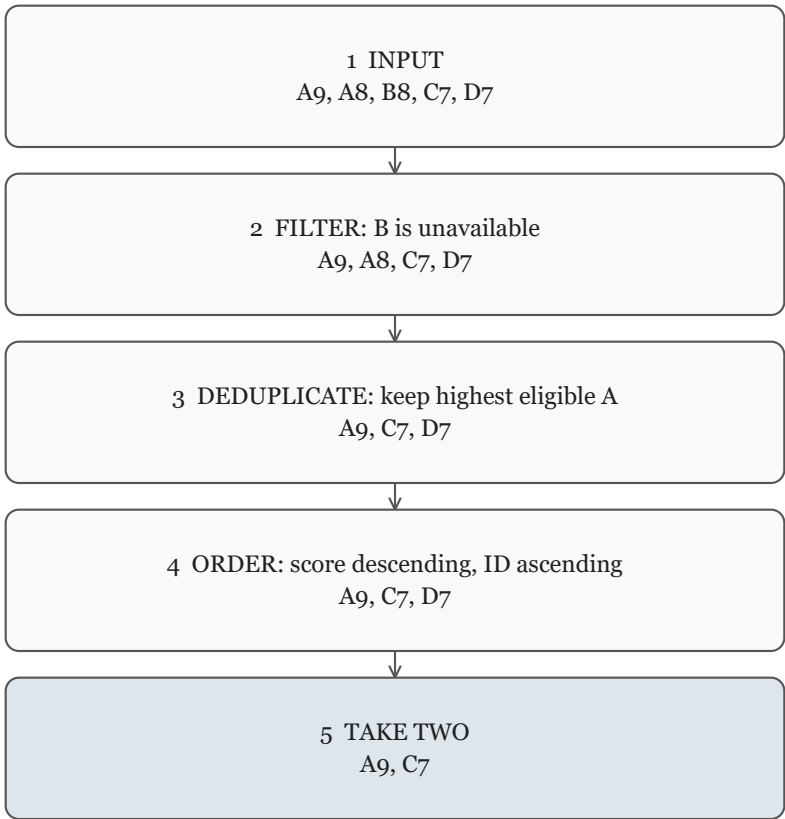
Temporal ranking reveals why a static top-k proof has limits. While processing a fixed set, once k better candidates dominate an outsider, discarding the outsider is safe. Later expiration can reduce the winners'

scores. The outsider may then deserve promotion, but a winner-only structure has forgotten it. An algorithm can be correct for static selection and insufficient for dynamic maintenance without any contradiction: the future operations changed what information must be retained.

Worked example: rank only after eligibility and identity are settled

Return the two highest-scoring eligible titles, breaking ties by smaller title ID. Records are A with score nine, another A with score eight, B with score eight but unavailable, C with score seven, and D with score seven. Define duplicate resolution as keeping the highest eligible score for each title.

RANKING PIPELINE: EACH STAGE HAS ONE JOB



Eligibility and duplicate resolution produce distinct candidates before the top-k comparison.

Stage	Remaining candidates	Reason
Raw records	A9, A8, B8, C7, D7	Five records, four identities
Eligibility	A9, A8, C7, D7	B unavailable
Deduplication	A9, C7, D7	Highest eligible A score wins
Ranking	A9, C7, D7	C precedes D on tied score
Top two	A9, C7	Two distinct eligible titles

Selecting the best two raw records would produce two copies of A. Deduplicating that result afterward would leave only one recommendation, even though valid alternatives existed. Similarly, taking top k before filtering can spend slots on unavailable titles. The pipeline order follows from what a result slot represents.

For a small candidate set, sorting the distinct eligible records is a clear baseline. A size-k heap saves work when the candidate set is large and k is small. Keep the worst retained candidate at the root so replacement compares against the current admission threshold. Apply the same score-and-ID rule at every stage; a final sort cannot recover a candidate that an inconsistent heap already discarded.

Worked example: diversity changes which candidates are useful

Suppose scores and genres are A9 drama, B8 drama, C7 comedy, and D6 documentary. Return three titles with at most one per genre.

Unconstrained top three is A, B, C. Removing repeated genres afterward leaves only A and C and loses the opportunity to select D.

For this simple constraint, with each title assigned exactly one genre and an independent scalar score, first keep the best eligible title per genre. The representatives are A9, C7, and D6. Rank those representatives to obtain three results. Replacing any lower-scoring choice from a genre

with its best representative cannot violate the one-per-genre constraint, which justifies this reduction.

That argument has limits. A title belonging to multiple constrained categories can compete for several budgets at once. Rules such as requiring two languages while limiting repeated actors interact across choices. Greedily taking the highest individual score can then block a better feasible combination. State the simpler contract rather than treating every recommendation task as the same heap exercise.

The domain story suggests possible requirements, but the coding task remains a sequence of precise operations: eligibility, canonical identity, aggregation if needed, comparison, and constrained selection. Keeping them separate makes a changed rule local and makes its correctness easier to explain.

Putting the program together: ranking eligible distinct titles

The function accepts candidate records, a watched-ID set, and k . Each candidate has ID, genre, integer score, and eligibility. It returns up to k distinct eligible unwatched titles, highest score first and smallest ID first on ties. For duplicate IDs, retain the highest-scoring eligible record; equal-score duplicates use the smaller genre string for determinism. Nonpositive k returns an empty result.

First scan candidates. Skip ineligible or watched records. Look up each remaining ID in a deduplication map. Keep the incoming record only if no record exists or it wins the duplicate-resolution rule. This stage establishes one canonical candidate per eligible ID under the exercise's policy. It must happen before selecting final winners.

For the baseline, collect the map values and sort using the final comparison. Return at most the first k . For a small k among many unique candidates, instead maintain a heap whose root is the worst retained winner. Push until it has k entries. Thereafter replace its root only when a newcomer is better, then repair the heap. Sort the retained winners into final output order before returning.

On A score eight, B nine, A ten, C nine, with B watched and k two, the canonical map contains A ten and C nine, so return A then C. If A and C tie, A still comes first. Test duplicate IDs, no eligible records, k larger than the result, and all equal scores. The comparison rule should give the same result regardless of map iteration order.

Expected scan cost is linear. Sorting all unique candidates costs $u \log u$; a bounded heap costs $u \log k$ plus sorting the final k. Both versions still retain the deduplication map, so their total auxiliary memory is not merely k. A one-per-genre rule changes selection: keep each genre's best representative before global selection, provided each title belongs to exactly one genre and the objective is the sum of these supplied scores.

Go example: write the ranking contract into the comparator

This complete function accepts candidates that are already eligible and unique by ID. It implements the sorting baseline, preserving the caller's slice. Include `import "sort"` with the file's imports. The copy is shallow, which is sufficient because these fields are a string and an integer.

```

type Candidate struct {
    ID string
    Score int64
}

func TopTitles(candidates []Candidate, k int) []Candidate {
    if k <= 0 {
        return nil
    }
    ranked := append([]Candidate(nil), candidates...)
    sort.Slice(ranked, func(i, j int) bool {
        a, b := ranked[i], ranked[j]
        if a.Score != b.Score {
            return a.Score > b.Score
        }
        return a.ID < b.ID
    })
    if k < len(ranked) {
        ranked = ranked[:k]
    }
    return ranked
}

```

For input D7, A9, C7 and k two, the result is A9, C7; the input remains D7, A9, C7. The comparator uses a strict less-than relation for IDs, never less-than-or-equal. Tied scores therefore get the promised ascending ID order. Eligibility and deduplication are preconditions, not missing steps that a sort can somehow infer.

Sorting u candidates costs $O(u \log u)$ comparisons and the copy uses $O(u)$ memory. Reslicing to k does not reduce the backing array's allocation. A heap can improve selection when k is small; begin with this clear baseline so the optimized implementation has an unambiguous result to match. The [sort.Slice documentation](#) defines its in-place behavior and comparator requirements.

Chapter 11 - Modeling larger practical programs

A practical prompt becomes manageable when you separate observable behavior, state, and transitions.

An in-memory filesystem

Directories have named children; files have contents. A tree mirrors this relationship. Resolve a path by splitting it into components and following child maps. Lookup costs $O(\text{number of components})$, plus string-processing cost. Listing a directory is proportional to its children, with additional sorting if the output must be lexical.

Before implementation, define whether repeated separators are allowed, whether dot and dot-dot are normalized, whether missing parents are created, and whether a file can be overwritten by a directory. You can deliberately restrict the first version to absolute normalized paths. State that restriction instead of silently implementing incomplete path semantics.

Move is not merely deleting one map entry and inserting another. Reject moving a directory inside its own descendant; otherwise you introduce a cycle or detach an entire subtree incorrectly. Validate destination rules before mutating either parent so failed moves leave the filesystem unchanged. Copy requires new nodes; sharing the old subtree would make edits to one copy change the other.

A transactional key-value store

Start with a base map and a stack of transaction overlays. Reads search overlays newest to oldest, then the base. Each overlay stores either a value or a tombstone representing deletion. A missing overlay entry means “keep searching”; a tombstone means “the key is absent.” Confusing those two states makes deleted values reappear from lower layers.

Begin pushes an empty overlay. Rollback discards the top overlay. Commit merges it into its parent overlay, or into the base if no parent exists. For nested transactions, an inner commit does not necessarily make changes durable outside the outer transaction. Our contract keeps them within the outer transaction until that transaction commits.

Trace base A equals five. Begin outer and set A to ten. Begin inner and set A to twenty. Commit inner: the outer overlay now holds twenty. Roll back outer: base A is five again. If your implementation committed the inner overlay directly to base, it violates this contract. Reads cost $O(\text{depth})$ map lookups; committing costs proportional to the top overlay's changes.

Failure case. Removing a key from an overlay cannot hide a value that still exists in a lower layer.

Removing the overlay entry exposes the older base value. A tombstone must stop lookup and report absence. This is the same distinction between “no update here” and “an explicit deletion” that appears in temporal storage and merge systems.

Pub/sub and callback ownership

Use topic to subscriber-ID to callback maps. Return a subscriber ID from Subscribe; arbitrary Go function values cannot be compared to each other for equality, so callback identity alone is not a suitable map key. On Publish, copy the current subscriber callbacks while holding the lock, release it, then invoke the copy. This allows callbacks to subscribe or unsubscribe without deadlocking or invalidating map iteration.

Define snapshot semantics: a subscriber removed during a publication may still receive that publication if it was in the snapshot. A synchronous slow callback delays later callbacks. Asynchronous delivery needs bounded buffers and a backpressure or drop policy; spawning unbounded goroutines transfers the problem into memory growth. Decide whether callback panics propagate or are recovered at the API boundary. These are behavioral choices with tests, not just implementation details.

A playback metadata service

Compose the previous lessons around `GetPlayback`. Validate user and video inputs. Check metadata cache. On a fresh hit, return the cached metadata. On a miss, join or initiate the per-key load. If loading succeeds, publish it with TTL and capacity accounting. If it fails, return an error or an explicitly allowed stale value. Expiration, eviction, and in-flight loading have different purposes.

Define metrics precisely. One request can be a cache miss while joining an already-running load, so backend-load count need not equal miss count. A stale value served under an outage policy needs its own metric if you care about freshness. Do not cache authorization decisions under video ID alone; user-specific data requires an appropriate key or a separate authorization check.

What if an object is larger than the memory budget? Return the successful backend result to the caller while declining to cache it under our chosen policy. Cache admission failure need not mean playback failure. What if twenty callers cancel while one remains? Shared load lifetime and individual waiting lifetimes are separate decisions.

Model a small program as a sequence of valid transitions

A larger coding prompt often overwhelms because it introduces several methods at once. Reduce it to a state machine: what is the state before a call, which preconditions allow the call, which mutations happen, and what must be true when it returns? You do not need formal notation. You need an operation sequence precise enough that another person could act as the computer.

For a filesystem move, the state consists of nodes and parent-child relationships. A successful move changes two relationships: remove the source from its old parent and add it under the destination parent. But checking whether the destination exists, whether the new name conflicts, and whether a cycle would be created should normally happen first. Otherwise a failed validation can leave the source detached. This is

the same “validate before destructive mutation” principle used for oversized cache admission.

A transaction overlay illustrates a different modeling skill: representing absence separately from no change. If an upper layer does not mention key A, read the lower layer. If the upper layer says A was deleted, stop and report missing. Both situations lack a normal value in the upper layer, but they have different behavior. A tombstone preserves that distinction. Many subtle bugs are really failures to represent two semantically different states separately.

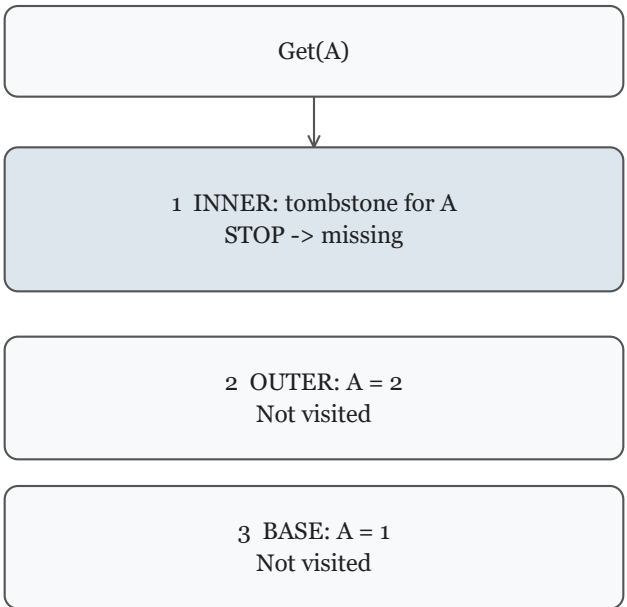
Callbacks make ownership and reentrancy visible. A pub/sub system owns its subscriber registry, but a callback is arbitrary user-controlled work. Holding the registry lock while executing that callback extends the lock across code whose duration and behavior you do not control. Taking a snapshot of subscribers separates the protected registry operation from later delivery. The snapshot policy then becomes part of the contract: a subscriber removed after the snapshot may still receive the current message.

Composition adds another dimension. A playback service can validate access, look up metadata, join an in-flight load, and update metrics. Keeping each responsibility explicit helps you reason about errors. A successful backend response that is too large to cache can still be returned. A cache miss that joins another caller's load need not cause another backend request. “Request failed,” “cache did not admit,” and “load was shared” should not collapse into one undifferentiated status.

Worked example: deletion is a value in the transaction overlay

The base map contains A:1. Begin an outer transaction and set A to two. Begin an inner transaction and delete A. A lookup must report missing, even though both lower layers still contain a value. The inner deletion is therefore represented by a tombstone, not by removing A from the inner map.

OVERLAY LOOKUP: DELETED IS NOT THE SAME AS ABSENT



Lookup searches from the newest transaction downward. A tombstone stops the search and reports missing.

Operation	Base	Outer overlay	Inner overlay	Visible A
Initial	A:1	None	None	1
Begin; Set A=2	A:1	A:2	None	2
Begin; Delete A	A:1	A:2	A:deleted	Missing
Commit inner	A:1	A:deleted	None	Missing
Roll back outer	A:1	None	None	1

Committing the inner layer copies its tombstone into the outer layer. It does not delete A from the base yet. Otherwise rolling back the outer transaction could not restore the original state. When the outermost

transaction commits, applying its tombstone finally removes A from the base.

Each overlay lookup has three outcomes: a value is present, a tombstone is present, or this layer says nothing. Only the third outcome permits searching older layers. This is a small instance of a broad modeling rule: absence of an instruction and an instruction to remove something are different states.

Worked example: define what a publish snapshot means

A topic currently has subscribers A and B. Publish copies the subscriber list under a lock, then releases the lock before calling callbacks. During A's callback, A unsubscribes B. The snapshot still contains B, so B receives this publication under snapshot semantics. Future publications omit B.

This is a reasonable contract, but it must be explicit. A stronger promise that unsubscribe prevents every later callback invocation would need additional coordination and a decision about a callback already in flight. Invoking arbitrary callbacks while holding the registry lock is a tempting shortcut that can deadlock when A calls unsubscribe on the same registry.

Event	Live registry	Current publish snapshot
Publish begins	A, B	A, B
A removes B	A	A, B
B callback runs	A	A, B
Next publish	A	A

Copying the subscriber list solves ownership of the list, not every failure policy. Synchronous callbacks mean one slow subscriber delays later ones. Asynchronous delivery requires queue bounds, cancellation, and

an overflow policy. A callback panic also needs an explicit policy if the prompt expects continued delivery. The first implementation should fulfill a small stated contract; extensions should name the new state and guarantee they introduce.

Putting the program together: nested transactions

The database exposes Begin, Get, Set, Delete, Commit, and Rollback. Outside a transaction, writes affect a base map. Inside a transaction, they affect only the newest overlay. An overlay entry is either a value or a deletion marker. Commit merges the newest overlay into its parent if one exists, otherwise into the base. Commit or Rollback with no active transaction returns an error without mutation.

Begin pushes an empty map onto a stack. Set stores a value entry in the top overlay, or the base when the stack is empty. Delete stores a tombstone in the top overlay, or removes the base entry outside a transaction. Get searches overlays newest to oldest. A value returns immediately, a tombstone returns missing, and no entry continues the search. Only after all overlays are silent does it read the base.

Commit first checks that an overlay exists. Remove that overlay from the stack, then merge each change into the new top layer. Copy both values and tombstones into a parent overlay so deletions remain explicit. When committing into the base, a tombstone performs an actual delete and a value performs an assignment. Rollback simply discards the newest overlay. These operations define nesting without needing to copy the entire database at each Begin.

With base A equal to five, Begin then Set A to ten affects only the outer overlay. Begin again and Set A to twenty affects the inner one. Commit inner moves twenty into outer. Rollback outer discards it, leaving base five. If inner Delete A is committed instead, the outer overlay must contain a tombstone; merely deleting A from the overlay would reveal the original five too early.

Get costs up to one map lookup per active depth. Commit work is proportional to the number of changes in the top overlay. This is a single-threaded nesting model, not an implementation of all database isolation levels. Adding concurrent transactions requires a visibility and conflict policy beyond simply putting a mutex around each method.

Go example: distinguish a tombstone from no instruction

This complete read operation searches transaction overlays from newest to oldest, then the base map. Each overlay maps keys to edits. A missing key, including in a nil overlay map, means that layer has no opinion; an explicit Deleted edit stops lookup. Writes and commit are separate operations described earlier.

```
type Edit struct {
    Value string
    Deleted bool
}

func ReadLayered(base map[string]string,
    layers []map[string>Edit, key string) (string, bool) {
    for i := len(layers) - 1; i >= 0; i-- {
        edit, found := layers[i][key]
        if !found {
            continue
        }
        if edit.Deleted {
            return "", false
        }
        return edit.Value, true
    }
    value, found := base[key]
    return value, found
}
```

For base A:1, outer A:2, and inner A:deleted, this returns missing immediately from the inner layer. Removing the inner layer reveals two. Removing both layers reveals one. Setting A to the empty string in the inner layer returns empty and true; the Deleted flag prevents that valid value from being mistaken for a tombstone.

The critical distinction occurs before the value is inspected: `found` says whether the layer has an instruction at all. Deletion is one such instruction. With d overlays, a lookup makes at most $d+1$ expected constant-time map lookups, so its expected time is $O(d)$, with constant extra space.

Chapter 12 - Complete interview case studies

These cases combine the book's ideas into programs that can be designed entirely in words. Read the brief, pause to reason if you wish, then follow the complete worked design. There is no requirement to type or execute code.

Case one: a metadata cache with recency and expiration

The brief is to support `Put(key, value, lifetime)` and `Get(key)` for video metadata, with a maximum number of resident entries. Reads must never return expired values. Successful reads and writes update recency. When there is no space, remove expired entries before evicting a live least-recently-used entry. A clock is provided so time can be controlled.

Resolve the observable rules

Choose these explicit rules for the case. Capacity is nonnegative; zero stores nothing. At the exact deadline, an entry is expired. A nonpositive lifetime removes the key. Overwriting a key changes its value, resets its deadline, and makes it most recent. `Get` does not extend the deadline. A missing `Get` does not change recency. A live value can be empty, so `Get` returns a found flag separately from the value.

These questions matter because they change branches and bookkeeping. They are not a request to design a globally distributed cache. Start with one process and one thread, then add concurrency after the state transitions are correct.

Build a correct baseline first

A map of entries plus a recency list can implement the baseline. Before a capacity-sensitive Put, scan the map and remove every entry whose deadline has passed. This is linear cleanup, but it is correct and easy to explain. Get checks only its own entry's deadline, so it need not scan the map. Identify cleanup as the expensive operation before introducing a heap to optimize it.

For the optimized version, the cache struct contains the key-to-node map, a doubly linked recency list, a deadline min heap, a generation sequence, capacity, and the clock. Each node contains key, value, deadline, generation, and previous/next links. Heap records contain key, deadline, and generation. The same live node participates in identity and recency; the heap can retain stale records that are checked before they act.

Define helpers around invariants

Detach joins neighboring list nodes. InsertFront adds a detached node immediately after the head marker. Remove unlinks a real node and deletes its map entry. If weights are later added, Remove also subtracts its weight exactly once. DrainExpired reads due heap records and removes a current node only when its generation matches.

Every current map entry must correspond to exactly one real list node. Every real node must have one map entry under its key. Neighbor links must agree. No expired node may be returned. On return from Put, resident count must be within capacity. Heap records are not required to correspond one-to-one with map entries in this lazy design, but stale records must never remove a newer generation.

Describe Get as an executable sequence

Look up the key. If absent, return missing. If present, read now and compare with the node's deadline. At or beyond the deadline, Remove the node and return missing. Otherwise Detach it, InsertFront it, and return its value with found true. A stale heap record may remain after lazy removal; later cleanup will see that its key is absent or its generation changed and ignore it.

This Get has expected constant map and list work. It does not perform global expiry cleanup. The guarantee is fresh visibility, not that a Get for one key instantly reclaims every other expired key.

Describe Put and the order of its changes

For zero capacity, retain nothing. For a nonpositive lifetime, remove an existing key and return. For a positive lifetime, read now and drain all due expirations before making a live capacity eviction. Find the key after cleanup. If it exists, update its value and deadline, assign a fresh generation, and promote it. If it does not, create and register a new node at the front. Push a matching deadline record. If count exceeds capacity, Remove the least-recent node at the tail.

After draining due records, count-limited insertion can require at most one live eviction. Draining itself can remove many records. Therefore the complete Put does not have a constant worst-case bound. Honest complexity names the work performed, including old heap records that do not delete anything.

Walk a boundary case

Capacity is two. At time zero, Put A with lifetime five and B with lifetime twenty. Recency is B, A. Get A makes it A, B. Advance to five and Put C with lifetime twenty. A has expired even though it is most recent. Cleanup removes A first. C can then be inserted without evicting B, leaving C, B.

At time six, overwrite C with lifetime fifty, so its new deadline is fifty-six. At time twenty-five, the old C deadline record is due. Its generation no

longer matches, so the new C survives. The same key name is not enough to authorize deletion.

Add concurrency and loading carefully

A first concurrency extension uses one mutex around each complete public operation. Get takes an exclusive lock because it promotes or removes nodes. The lock protects map/list/generation consistency, not just the map. Helpers assume the caller holds the lock rather than locking recursively.

If metadata is loaded from a slow service on a miss, do not hold this global mutex through the service call. Create or join per-key in-flight work under the lock, release the lock while fetching, then publish or complete the result under a protected transition. Release waiters on both success and failure. If invalidation occurred during the fetch, a separate generation check can prevent an obsolete result from repopulating the key.

Tests you can reason through without running

Capacity zero must remain empty. A stored empty value must still be a hit. Expiry exactly at the deadline must miss. Refresh before an old deadline must survive old cleanup. Overwrite must not create duplicate list nodes. An expired most-recent entry must be removed before a live least-recent eviction under this policy. Repeated refreshes must be included in memory analysis because their stale heap records may accumulate.

These checks cover visibility, structural consistency, and lifecycle identity. Separating those categories helps locate a missing invariant when a requirement changes.

Case two: recent viewing totals and ranked recommendations

The brief is to accept a finite collection of viewing events and return the top k titles by total watch duration in a recent window. Each event has event ID, user ID, title ID, duration, and event timestamp. Redeliveries with the same event ID count once. Tied totals rank by ascending title ID. The caller supplies now and window width.

Define the batch contract

Use the interval strictly after now minus width and at or before now. Future events are excluded. The first version assumes repeated IDs have identical payloads. Durations are positive; choose to reject invalid duration records rather than let them silently affect totals. k at or below zero gives an empty result. Input order is arbitrary and preserved. This is a batch query, so event order need not support a queue.

The program separates validation, filtering, identity, aggregation, and ranking. This separation helps you explain a change without rewriting the whole solution. If the prompt wants invalid records skipped instead of rejected, that changes validation policy while leaving the valid-data aggregation argument intact.

Choose fields and helper responsibilities

A seen-event set suppresses duplicates. A totals map accumulates duration by title. A result record contains title ID and total duration. A comparison helper orders larger totals first, then smaller IDs. The main function validates records, applies the window rule, suppresses duplicates among relevant identical deliveries, and adds each accepted contribution to the totals map.

Because duplicate payloads are assumed identical, filtering an out-of-window delivery before deduplication cannot hide an in-window version of that same event. If duplicate IDs can have different timestamps or values, this reasoning fails. You then need a canonical record policy, conflict rejection, or a defined correction model. State that changed requirement before modifying the code structure.

Use a simple ranking baseline

Collect the totals map into a slice of result records. Sort by the complete comparison and return up to k . If n is events and u is distinct qualifying titles, expected scan work is linear in n , sorting is $u \log u$, and retained identity plus aggregation state can be linear in n plus u . A map being fast does not make its memory free.

For small k , use a heap containing at most k winners and exposing the worst retained one. Compare every unique title's final total with that root. Sort the surviving winners before returning. This saves ranking work but does not remove the seen-event set or totals map. Do not claim the entire algorithm now needs only k memory.

Execute the example mentally

At now ten with width five, event e_1 gives A three duration at time six. Event e_2 gives A four at nine and is delivered twice. Event e_3 gives B seven at ten. Event e_4 gives C one hundred at five. The interval excludes time five, so C contributes nothing. A totals seven because e_2 counts once. B also totals seven. A wins the ID tie, so the ranking is A followed by B.

An event at eleven is in the future and is excluded. An empty collection gives no titles. k larger than the number of qualifying titles returns all of them without inventing missing positions. These observations follow directly from the contract and can be checked without executing a program.

Change the input to a stream

Now events arrive in nondecreasing timestamp order, and queries also move forward in time. Retain a queue of accepted contributions. Before answering at now, remove contributions at or before the cutoff and subtract their durations from their titles' aggregates. Maintain counts as well as sums if needed to distinguish a title with no remaining events. Delete an aggregate when its last retained event leaves.

The queue works because expiry order matches event-time order. If late events are allowed, insertion order no longer proves that expired

contributions form a prefix. Define a bounded-lateness policy, an ordered structure, or a separate event-time processing rule. Do not silently accept arbitrary late events into a FIFO queue.

Deduplication retention is not automatically identical to the ranking window. If the sender can redeliver an old event after its identity marker has expired, the program needs to decide whether the event is still eligible and whether it can be counted again. For corrections with reused IDs, you may need to remove an old contribution before adding its replacement. Identity, event eligibility, and processing lifetime remain separate decisions.

Understand score decreases

Suppose A is the current winner and B is second. A's oldest contribution expires, reducing its score below B. A heap containing only A cannot discover B if B has been discarded from all other state. Keep full active aggregates and rank on query for a correct baseline, or maintain an indexed ordered representation covering all candidates. A static top-k selection proof cannot be reused unchanged for scores that decrease.

Explain the final program to an interviewer

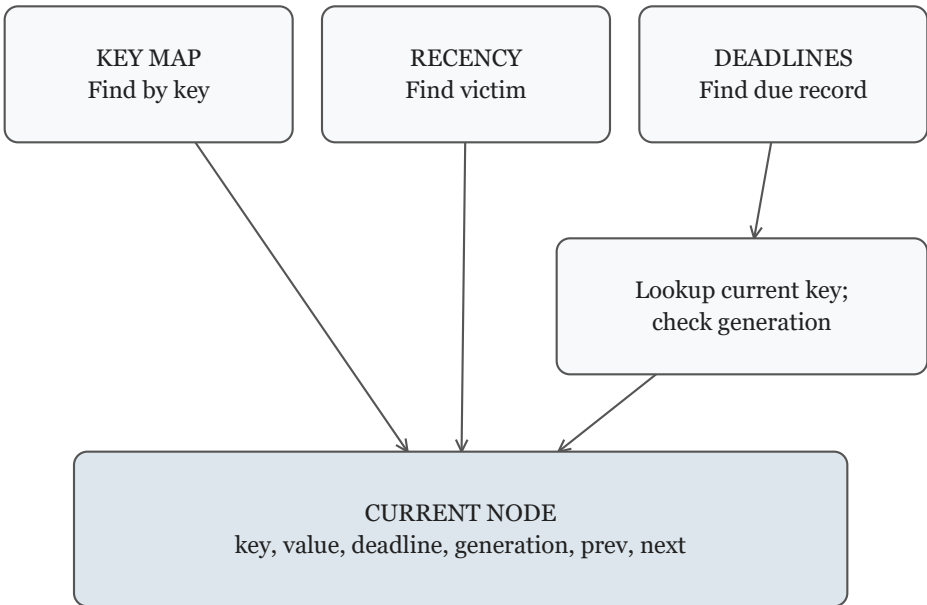
Begin with the batch contract and the simplest correct scan-and-sort program. State which map captures identity and which captures aggregation. Show the boundary example and deterministic tie rule. Then explain the small-k optimization and its true memory bound. For streaming, introduce reversible contributions, ordered expiry, and the limitation of winner-only state. Each extension has a specific changed operation and invariant.

Visual synthesis: three indexes, one cache entry

The combined cache case has three different reasons to locate an entry. A Get uses key identity. Capacity management uses recency. Expiration cleanup uses deadline order. The diagram separates those queries while keeping one authoritative current entry per key.

THREE QUERIES, ONE AUTHORITATIVE RESIDENT

Map/list refer to current nodes; heap records may be stale.



Three views cooperate: the key map finds the current node, the recency list finds a capacity victim, and the deadline heap proposes expiry candidates.

The map and recency list have one-to-one membership. The heap deliberately does not: refreshes can leave stale deadline records. Treating every heap record as a resident entry would make the implementation and its space analysis disagree. Generation checking is the bridge from a historical deadline record to the current node.

Worked example: expiration wins over recency

Capacity is two. At time zero write A with deadline five and B with deadline twenty. Read A, making it most recent. At time five, insert C with deadline twenty-five. A is expired, so cleanup removes A before any live capacity eviction. B survives even though B was least recent just before the operation.

Stage at time 5	Recency, newest first	Live residents	Action
Before cleanup	A, B	B	A expired but still stored
After cleanup	B	B	Remove A from map and list
After inserting C	C, B	C, B	No live eviction needed

This trace distinguishes two independent meanings of old: old by last access and old by deadline. A single ordered list generally cannot answer both questions. It also illustrates why observable policy determines operation order. Evicting B before cleaning A would satisfy the entry-count limit but violate this case's expired-first rule.

Worked example: reverse a contribution before reranking

For the viewing-statistics case, use a ten-second window and three events: e1 contributes four to A at time two; e2 contributes six to B at time eight; e3 contributes three to A at time nine. A repeated delivery of e2 contributes nothing extra. At time nine, totals are A:7 and B:6, so A ranks first.

At time twelve, e1 falls on the excluded left boundary. Subtract four from A, leaving A:3 and B:6; B now ranks first. A heap containing only the former winner A cannot infer this change from its own state. A simple correct live implementation retains all current title totals and recomputes ranking when queried. An optimized design needs an update mechanism that keeps ranking synchronized with both additions and expirations.

Query time	A total	B total	Winner
9	7	6	A
12	3	6	B
18	3	0	A
19	0	0	No active title

At time eighteen, the event at eight is excluded; at nineteen, the event at nine is excluded. These equalities are useful tests because they expose inconsistent boundary rules between ingestion, expiration, and querying. Combining components successfully means their contracts agree, not merely that each component works in isolation.

Go example: compose filtering, identity, and aggregation

This complete batch aggregation core assumes positive durations, identical payloads for duplicate IDs, a positive window, and arithmetic fitting int64. Invalid-input validation belongs before this core. Input order is arbitrary. Its output is the totals map consumed by the ranking stage, not the final top-k list.

```

type WatchEvent struct {
    ID, Title string
    At, Minutes int64
}

func RecentTotals(events []WatchEvent,
    now, window int64) map[string]int64 {
    seen := make(map[string]bool)
    totals := make(map[string]int64)
    for _, event := range events {
        if event.At <= now-window || event.At > now {
            continue
        }
        if seen[event.ID] {
            continue
        }
        seen[event.ID] = true
        totals[event.Title] += event.Minutes
    }
    return totals
}

```

For e1=A4 at two, e2=B6 at eight, e3=A3 at nine, and a repeated e2, querying at twelve with width ten returns A:3 and B:6. e1 is exactly on the excluded left boundary. The second e2 is ignored before aggregation, so B does not become twelve. A future event is excluded even if its ID has not appeared before.

Filtering before deduplication is safe here because duplicate payloads are identical. If an ID can arrive with a corrected timestamp or duration, a canonical-version rule must precede this core. The expected work is linear in input events, with space for distinct accepted event IDs and titles. Unlike the streaming design, a new batch query rebuilds these summaries and therefore needs no inverse operation for expiration.

Sources and scope

Research checked on 30 September 2026. The exercises and learning sequence are original preparation material; they are not a leaked or verified Netflix question bank.

What informed the preparation

Your plan.md is the primary scope input. It supplies the core algorithm sequence and practical topics, particularly LRU, TTL, rate limiting, event processing, and iterative requirements. Your clarification establishes an L5 live interview with another senior engineer. No interview date or team-specific preparation guide was supplied, and no chapter requires a particular study duration.

Internet searches covered Netflix technical-screen descriptions, CodeSignal's own documentation, recommendation-system sources, and Go references. Search results contained conflicting commercial guides and secondhand reports. Those reports were not treated as proof of what you will be asked. The live format and level come from you, not from those reports.

Primary sources used

- [CodeSignal live interviewing](#) supports the distinction between an interviewer-led session and an assessment. It does not specify Netflix's content or evaluation rubric.
- [CodeSignal practice overview](#) supports practising the IDE and question formats. Your invitation remains the source for session-specific constraints.
- [CodeSignal language support](#) lists platform language support visually. We do not infer a particular Go runtime version from it; verify that in your actual environment.

- [Netflix recommendation overview](#) supplies high-level product context, not a coding interview syllabus.
- [Google candidate generation overview](#) explains content-based and collaborative retrieval. The toy ranking formulas and tasks here are our own.
- [Go container/heap](#) and [sort.Search](#) support heap interface and binary-search API details. Current source documentation was also fetched through Context7 using `/golang/go`.
- [Go sync](#), [memory model](#), and [time](#) support synchronization and clock discussion.
- [Go specification](#) and [strings, bytes, runes and characters](#) support language behavior.
- [Go test command](#) and [race detector](#) explain local validation tools. Race detection checks executed paths and does not prove absence of all races.

Coverage of the original plan

Two Sum, anagrams, windows, and array products appear in lesson 2. Three Sum, intervals, binary search, top k, and kth-element selection share lesson 3's ordering tools. Tree traversal, BST validation, islands, course dependencies, and graph cloning appear in lesson 4. LRU, weighted capacity, TTL, combined expiration, and concurrency are lessons 6 and 7. The two rate limiters are lesson 8. Time-based storage, deduplication, counters, and scheduling are lesson 9. Streaming statistics and recommendation tasks are lesson 10. Filesystem, transactions, pub/sub, and playback-service composition are chapter 11 and its worked program design.

Not every topic has a complete code solution. The supplied runnable references cover representative patterns and the highest-priority cache and limiter building blocks. Other tasks include derivations, traces, hints, contracts, and suggested tests so that you practise constructing them. Two original case studies combine several ideas and supply complete worked designs. An AI tutor can also use them as oral mocks.

A further connection

For k th largest in a stream, use a size- k min heap over all seen values; duplicates count as separate observations unless specified otherwise. For a static array, sorting is a simple baseline and quickselect offers expected linear time, but its worst case and pivot handling require care. For this first-round route, reliable heap reasoning is the higher-priority transferable skill.

Limits of the material

This course provides focused preparation, not exhaustive coverage of every possible interview problem. Deep distributed systems, advanced ML training, and a full Go language course are outside this first coding-round scope. The book can be read independently without an editor, running tests, or an AI. Spoken implementations develop reasoning but are not evidence that code has compiled.

Repository and conversation access

The book is independent of a particular ChatGPT interface. For the repository workflow, an agent must actually retrieve the relevant files. Do not infer that remote read access includes permission or tools to update progress. [Official ChatGPT Voice documentation](#) describes conversational turn-taking and task permissions. [Official workspace connection guidance](#) distinguishes resource access from write actions and recommends checking them separately. These pages do not verify this reader's account configuration.